

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
import scipy as sp
import pandas as pd
```

Datasets

Temper

```
In [ ]: df_temper = np.load("temper.npz")
```

```
In [ ]: # Display "villes"

temper_villes = df_temper["villes"]
```

```
In [ ]: # Set data in pandas DataFrame

temper_data_np = df_temper["data"][:,12]
temper_data = pd.DataFrame(temper_data_np, index = temper_villes)
temper_data
```

```
Out [ ]:
```

	0	1	2	3	4	5	6	7	8	9	10	11
Bordeaux	5.6	6.6	10.3	12.8	15.8	19.3	20.9	21.0	18.6	13.8	9.1	6.2
Brest	6.1	5.8	7.8	9.2	11.6	14.4	15.6	16.0	14.7	12.0	9.0	7.0
Clermont-Ferrand	2.6	3.7	7.5	10.3	13.8	17.3	19.4	19.1	16.2	11.2	6.6	3.6
Grenoble	1.5	3.2	7.7	10.6	14.5	17.8	20.1	19.5	16.7	11.4	6.5	2.3
Lille	2.4	2.9	6.0	8.9	12.4	15.3	17.1	17.1	14.7	10.4	6.1	3.5
Lyon	2.1	3.3	7.7	10.9	14.9	18.5	20.7	20.1	16.9	11.4	6.7	3.1
Marseille	5.5	6.6	10.0	13.0	16.8	20.8	23.3	22.8	19.9	15.0	10.2	6.9
Montpellier	5.6	6.7	9.9	12.8	16.2	20.1	22.7	22.3	19.3	14.6	10.0	6.5
Nantes	5.0	5.3	8.4	10.8	13.9	17.2	18.8	18.6	16.4	12.2	8.2	5.5
Nice	7.5	8.5	10.8	13.3	16.7	20.1	22.7	22.5	20.3	16.0	11.5	8.2
Paris	3.4	4.1	7.6	10.7	14.3	17.5	19.1	18.7	16.0	11.4	7.1	4.3
Rennes	4.8	5.3	7.9	10.1	13.1	16.2	17.9	17.8	15.7	11.6	7.8	5.4
Strasbourg	0.4	1.5	5.6	9.8	14.0	17.2	19.0	18.3	15.1	9.5	4.9	1.3
Toulouse	4.7	5.6	9.2	11.6	14.9	18.7	20.9	20.9	18.3	13.3	8.6	5.5
Vichy	2.4	3.4	7.1	9.9	13.6	17.1	19.3	18.8	16.0	11.0	6.6	3.4

```
In [ ]: temper_mean = temper_data.mean(axis = 1)
```

```
In [ ]: # Set citie's coordinates in pandas DataFrame

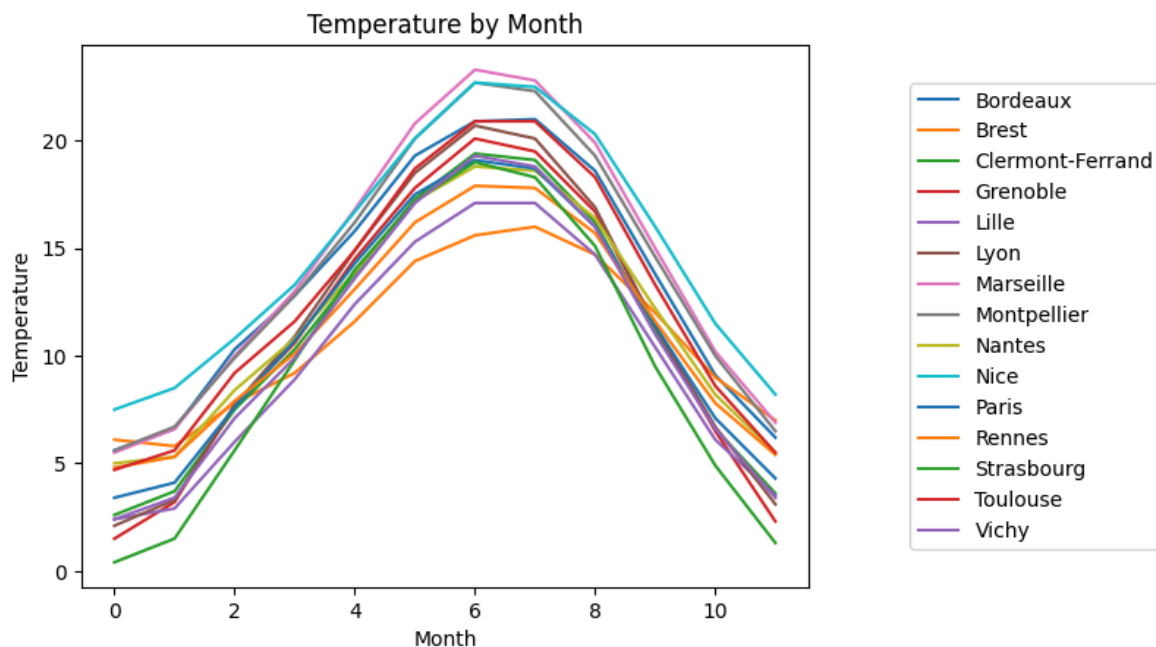
city_coords_np = df_temper["data"][ :,12:]
city_coords = pd.DataFrame(city_coords_np, index = temper_villes, columns
city_coords
```

```
Out [ ]:           Latitude Longitude
```

Bordeaux	44.5	-0.34
Brest	48.2	-4.29
Clermont-Ferrand	45.5	3.05
Grenoble	45.1	5.43
Lille	50.4	3.04
Lyon	45.5	4.51
Marseille	43.2	5.24
Montpellier	43.4	3.53
Nantes	47.1	-1.33
Nice	43.4	7.15
Paris	48.5	2.20
Rennes	48.1	-1.41
Strasbourg	48.4	7.45
Toulouse	43.4	1.26
Vichy	46.1	3.26

```
In [ ]: # Display Temperature Data

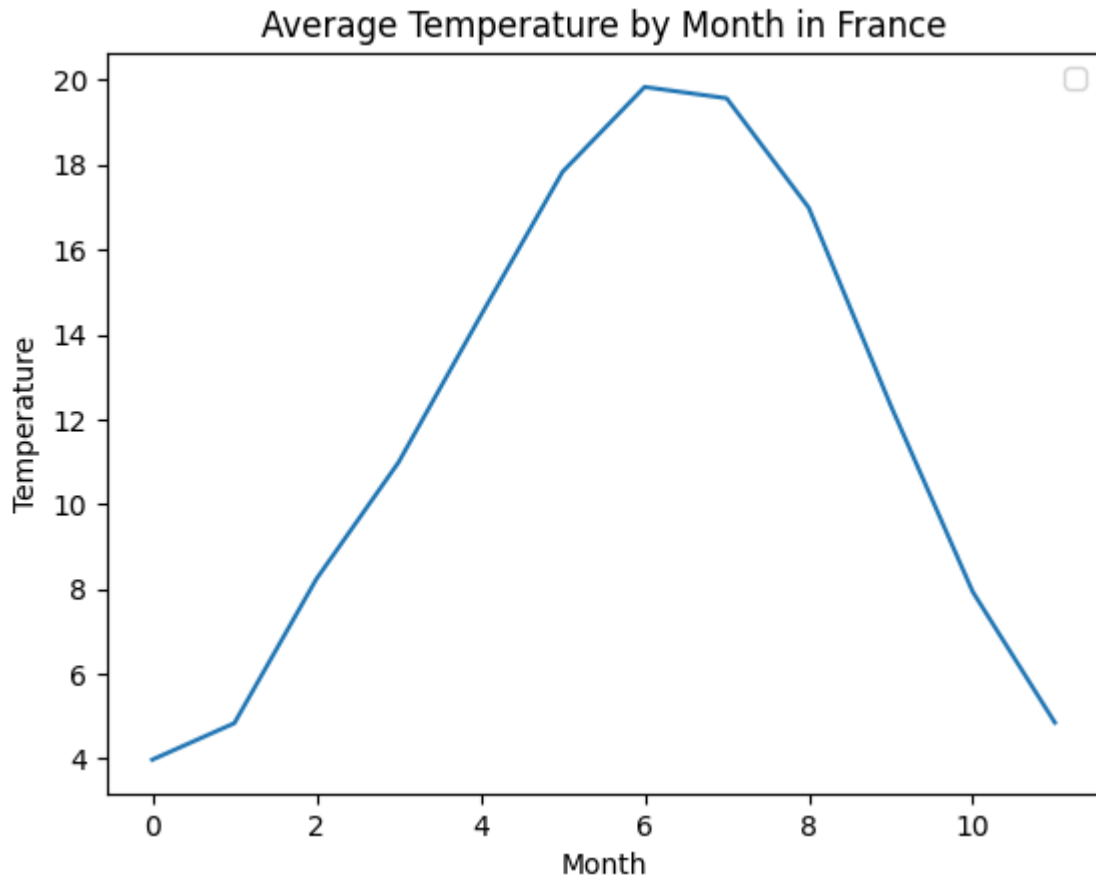
plt.figure()
for city in temper_villes :
    plt.plot(temper_data.columns, temper_data.loc[city], label = city)
plt.xlabel("Month")
plt.ylabel("Temperature")
plt.title("Temperature by Month")
plt.legend(loc = "right", bbox_to_anchor = (1.5,0.5))
plt.show()
```



In []: *# Display Average Temperature for each Month*

```
plt.figure()
plt.plot(temper_data.columns, temper_data.mean(axis = 0))
plt.xlabel("Month")
plt.ylabel("Temperature")
plt.title("Average Temperature by Month in France")
plt.legend()
plt.show()
```

/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/3552384404.py:8: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend () is called with no argument.
plt.legend()



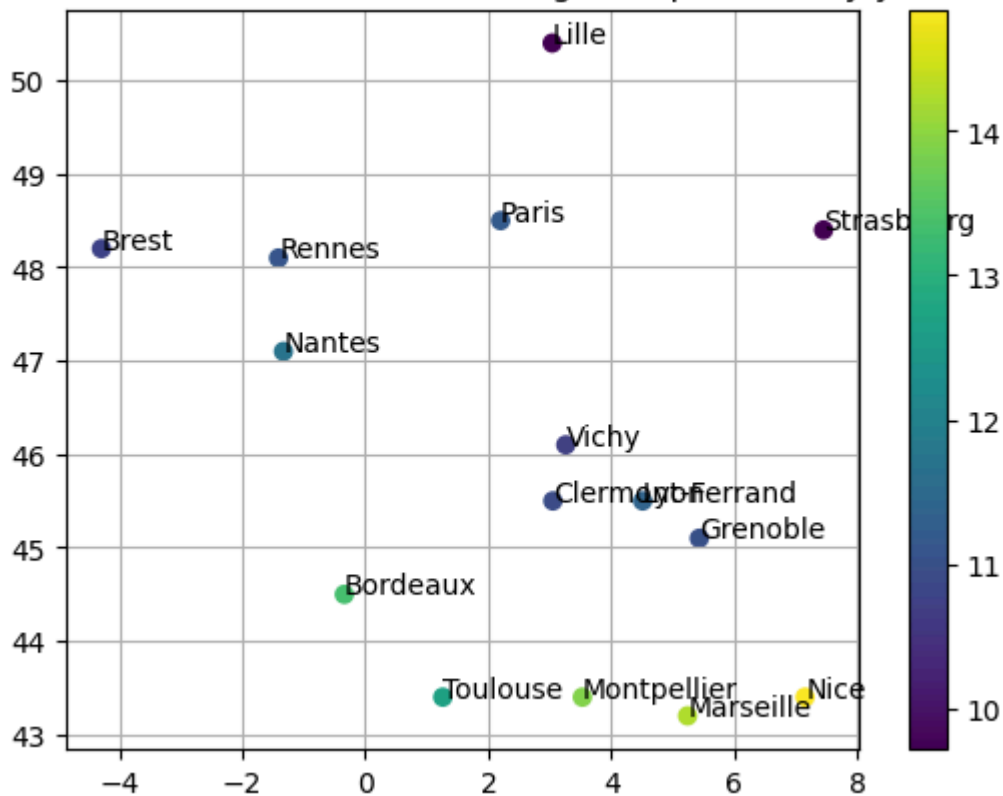
In []: *# Display Cities*

```
def display_cities(temper_mean) :
    plt.figure()
    x, y = city_coords.Longitude, city_coords.Latitude
    plt.scatter(x, y, c = temper_mean, cmap = 'viridis')
    for i in range(len(x)) :
        plt.annotate(np.array(city_coords.index)[i], (x[i], y[i]))
    plt.grid(True)
    plt.colorbar()
    plt.title("Position of cities in France + average temperature by year")
    plt.show()
```

In []: `display_cities(temper_data.mean(axis = 1))`

```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/127493545
7.py:8: FutureWarning: Series.__getitem__ treating keys as positions is de
precated. In a future version, integer keys will always be treated as labe
ls (consistent with DataFrame behavior). To access a value by position, us
e `ser.iloc[pos]`
    plt.annotate(np.array(city_coords.index)[i], (x[i], y[i]))
```

Position of cities in France + average temperature by year



Digits

```
In [ ]: digit_data = np.load("digits.npz")
digit_data
```

```
Out[ ]: NpzFile 'digits.npz' with keys: xt, yt, y, x
```

xt, yt, x and y contains the same kind of data, let's visualize some points of xt

```
In [ ]: xt = digit_data["x"]

# Let's regularize the data :
xt = xt.astype(np.float64)/255

rd_idx = np.random.randint(0, len(xt), 45)

imgs = xt[rd_idx]
rd_idx
```

```
Out[ ]: array([1620, 339, 1610, 954, 1559, 2967, 397, 1424, 2341, 1416, 124,
        435, 739, 2813, 945, 2156, 82, 2611, 1722, 660, 1608, 1179,
        892, 1937, 1013, 2511, 1840, 1609, 2002, 1142, 1915, 2270, 1349,
        2246, 800, 560, 211, 1782, 2129, 1900, 1341, 1466, 2536, 2626,
        171])
```

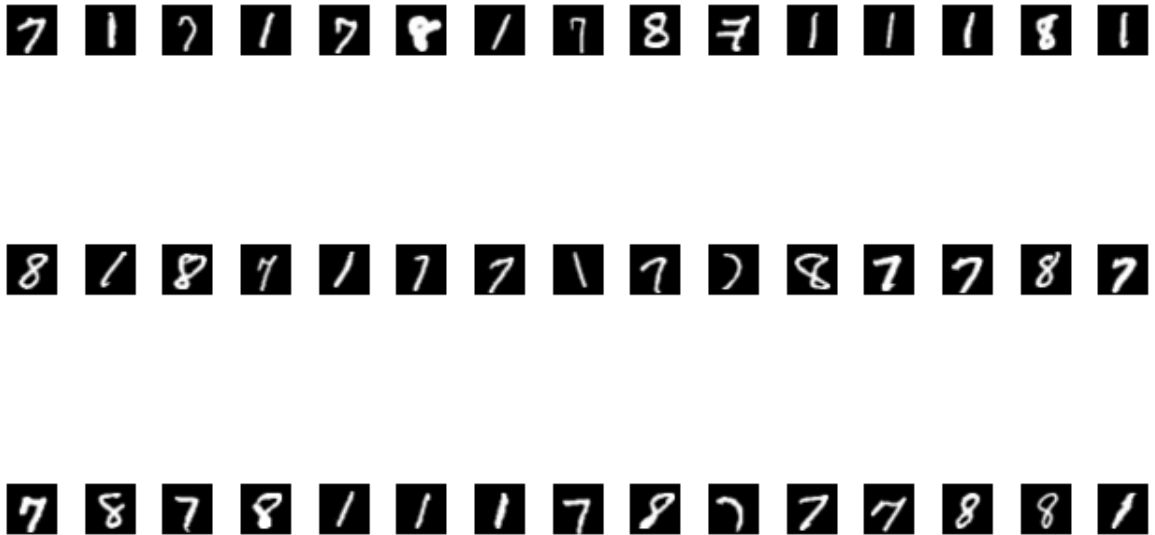
```
In [ ]: ## Defining a function to plot images
def display_images(imgs):
    fig, axs = plt.subplots((len(imgs)-1)//15 +1 ,min(len(imgs), 15))
    for i in range(len(imgs)):
        img = [imgs[i][t*28:(t+1)*28] for t in range(28)]
        if (len(imgs)-1)//15 > 0 :
```

```

        axs[i//15, i%15].axis('off')
        axs[i//15, i%15].imshow(img, cmap = "gray")
    else :
        axs[i%15].axis('off')
        axs[i%15].imshow(img, cmap = "gray")
plt.tight_layout()
plt.show()

```

```
In [ ]: display_images(imgs)
```



Clustering

```
In [ ]: from sklearn import cluster
```

Temperature

```
In [ ]: kmeans = cluster.KMeans(3, random_state=42)
```

```
In [ ]: temper_kmeans = kmeans.fit(temper_data)
temper_clusters = temper_kmeans.labels_
cmap = ['orange', 'g', 'c', 'r', 'b']
```

```
In [ ]: temper_centers = temper_kmeans.cluster_centers_
```

```
In [ ]: # Display cluster geographically

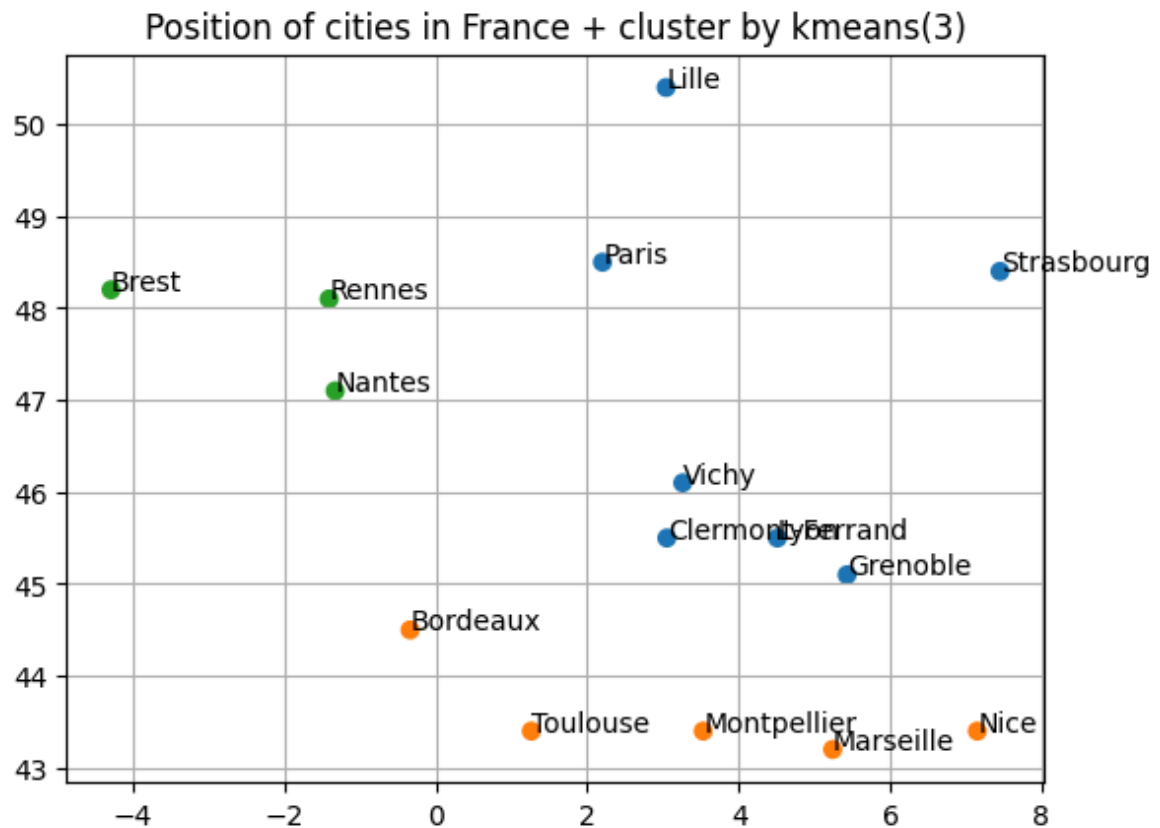
plt.figure()
x, y = city_coords.Longitude, city_coords.Latitude
plt.scatter(x, y, c = temper_clusters, cmap = 'tab10', vmax = 9)
for i in range(len(x)) :
    plt.annotate(np.array(city_coords.index)[i], (x[i], y[i]))
plt.grid(True)
plt.title("Position of cities in France + cluster by kmeans(3)")
plt.show()

```

```

/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/364482710
0.py:7: FutureWarning: Series.__getitem__ treating keys as positions is de
precated. In a future version, integer keys will always be treated as labe
ls (consistent with DataFrame behavior). To access a value by position, us
e `ser.iloc[pos]`
plt.annotate(np.array(city_coords.index)[i], (x[i], y[i]))

```



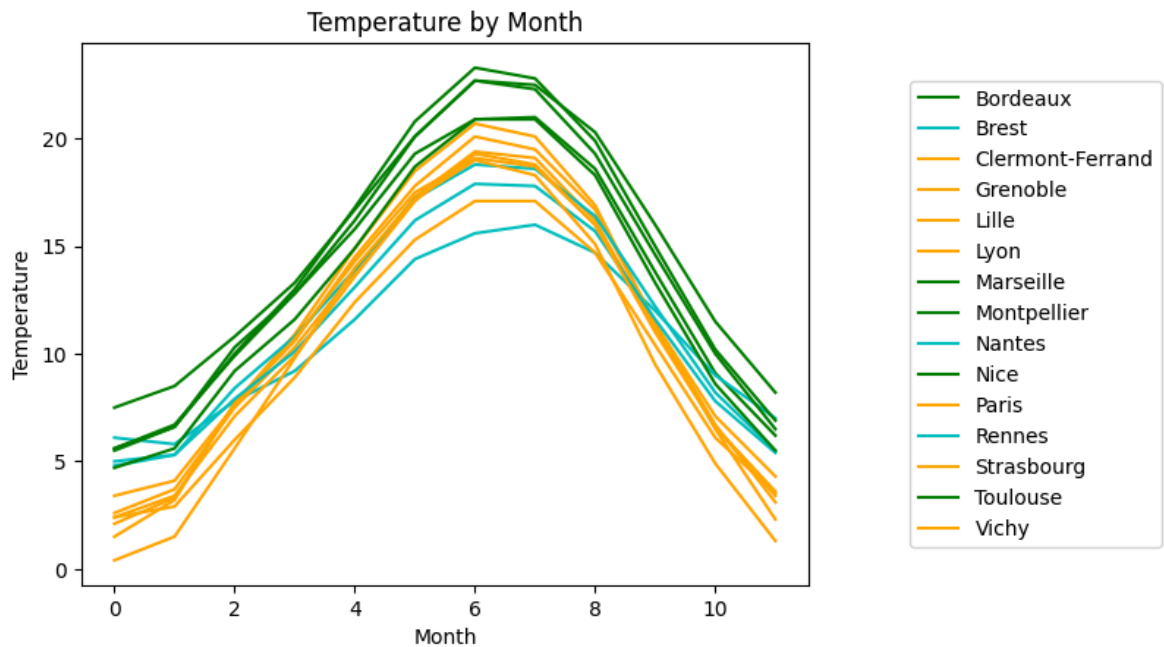
We can observe that the kmeans clustering is separating the cities in three delimited geographical groups and that the orange cluster is regrouping the warmest cities by comparing with the precedent geographical display. Also the green and blue clusters seems undifferentiable regarding average temperature through the year.

```
In [ ]: temper_clusters_bycity = {key : value for key, value in zip(temper_villes
```

```
In [ ]: # Display Temperature colored by clusters

plt.figure()
for city in temper_villes :
    plt.plot(temper_data.columns, temper_data.loc[city], label = city, c
plt.xlabel("Month")
plt.ylabel("Temperature")
plt.title("Temperature by Month")
plt.legend(loc = "right", bbox_to_anchor = (1.5,0.5))
plt.show()

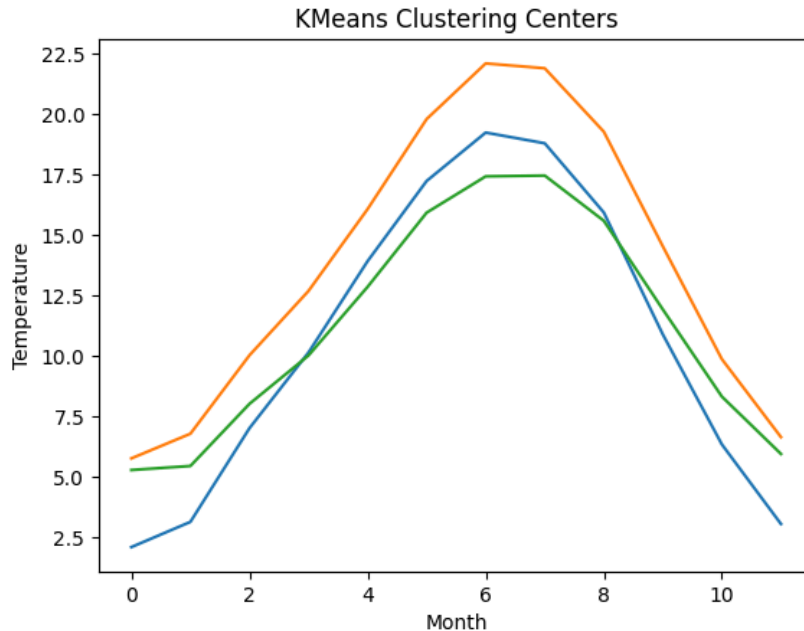
```



```
In [ ]: def display_profiles(curves, monthes = temper_data.columns, labels = None
        kwargs = {}
        if cmap is not None :
            kwargs["c"] = cmap
        for i in range(len(curves)) :
            if labels is not None :
                kwargs["label"] = labels[i]
            plt.plot(monthes, curves[i], **kwargs)
        plt.xlabel("Month")
        plt.ylabel("Temperature")
        plt.title("Temperature by Month")
        plt.legend(loc = "right", bbox_to_anchor = (1.5,0.5))
```

```
In [ ]: # Display Cluster centers
display_profiles(temper_centers)
plt.title("KMeans Clustering Centers")
plt.show()
```

```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/324712143
9.py:12: UserWarning: No artists with labels found to put in legend. Note
that artists whose label start with an underscore are ignored when legend
() is called with no argument.
plt.legend(loc = "right", bbox_to_anchor = (1.5,0.5))
```

Within this graph we can observe that the blue and green clusters differentiate by the variance of their temperature through the year. By putting this observation in perspective with the geo can guess that blue cluster correspond to a continental climate while the green one corresponds to an oceanic climate.

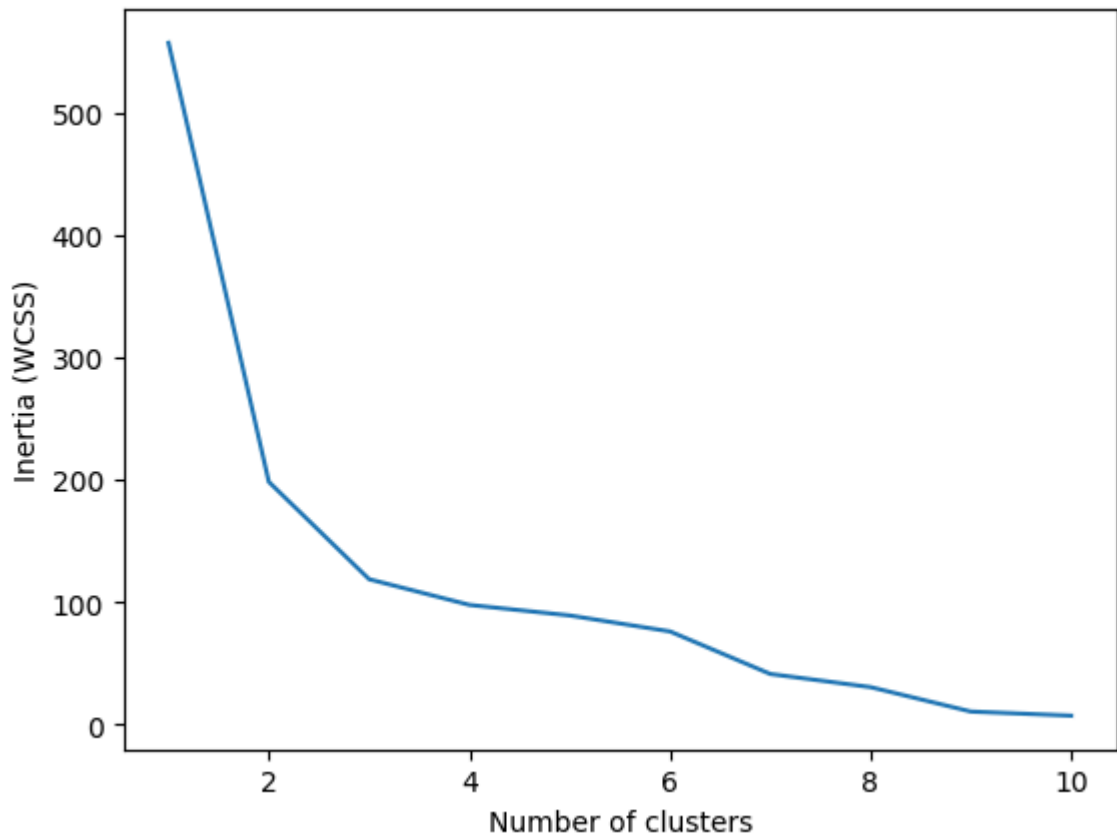
Now let's determinate the optimal number of clusters using the elbow method. This method requires to fit kmeans on the data for increasing values of the number of clusters and observe the evolution of the inertia of resulting clusters

```
In [ ]: Kmax = 11
inertia = []
centers = []
clusters = []
```

```
In [ ]: ## Compute and display inertia

for i in range(1, 11) :
    kmeans = cluster.KMeans(i, random_state = 42)
    kmeans.fit(temper_data)
    inertia.append(kmeans.inertia_)
    centers.append(kmeans.cluster_centers_)
    clusters.append(kmeans.labels_)
```

```
In [ ]: plt.figure()
plt.plot(np.arange(1,11), inertia)
plt.xlabel("Number of clusters")
plt.ylabel("Inertia (WCSS)")
plt.show()
```



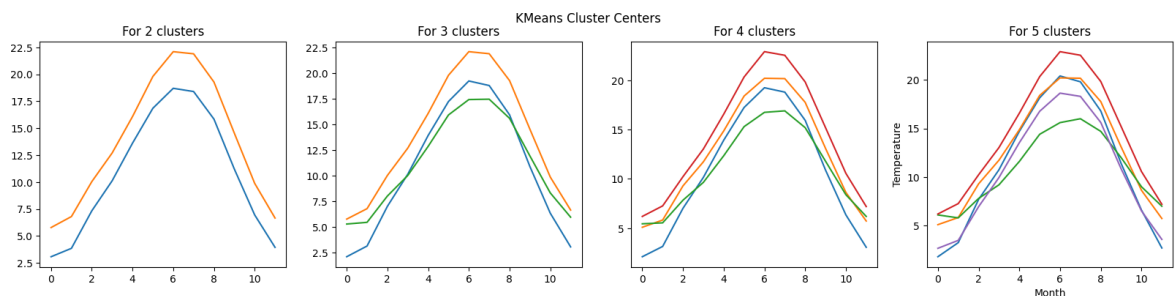
K = 3 seems to be the optimal value of the temperature data

To make sure the choice we made is the good one, let's display temperature means for K = 2 and K = 5

```
In [ ]: fig, axs = plt.subplots(1,4, figsize = (20,4))

for i in range(1,5):
    centers_temp = centers[i]
    for j in range(len(centers_temp)) :
        axs[i-1].plot(temper_data.columns, centers_temp[j])
        axs[i-1].set_title(f"For {i+1} clusters")

plt.suptitle("KMeans Cluster Centers")
plt.xlabel("Month")
plt.ylabel("Temperature")
plt.show()
```



We can observe that :

- For two clusters all the coldest cities are in the same cluster, that is quite a rough simplification of the different temperature variation present in these cities

(visible on the centers of the 3 clusters).

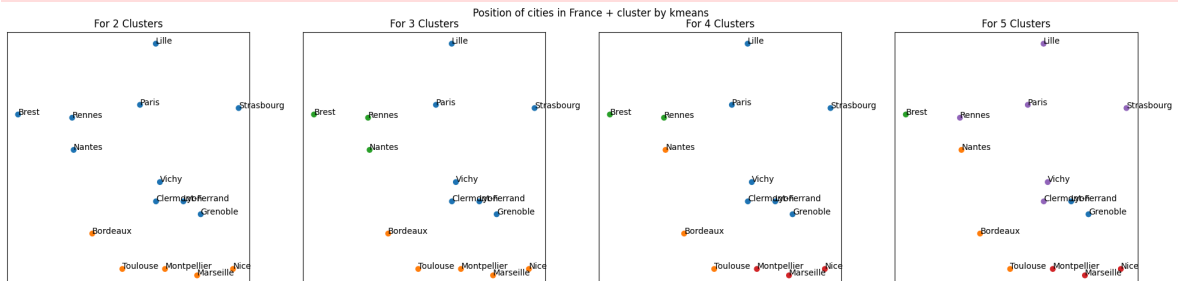
- For four or more clusters, even if some clusters are close to be a translation of an other cluster, some differences seems to appear on the middle of the curve (for example in the 4 and 5 clusters curves, the red curve seems to be sharper than the orange). Discerning these differences don't seem to have a huge impact on the inertia but could be relevant to have a more precise clustering regarding the profile of temperature.

```
In [ ]: # Display cluster geographically

fig, axs = plt.subplots(1,4, figsize = (20,5))

x, y = city_coors.Longitude, city_coors.Latitude
for i in range(1,5) :
    axs[i-1].scatter(x, y, c = clusters[i], cmap = 'tab10', vmax = 9)
    axs[i-1].set_title(f"For {i+1} Clusters")
    axs[i-1].set_xticks([])
    axs[i-1].set_yticks([])
    for j in range(len(x)) :
        axs[i-1].annotate(np.array(city_coors.index)[j], (x[j], y[j]))
plt.tight_layout(pad=2.0)
plt.suptitle("Position of cities in France + cluster by kmeans")
plt.show()
```

```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/135390955
4.py:12: FutureWarning: Series.__getitem__ treating keys as positions is d
eprecated. In a future version, integer keys will always be treated as lab
els (consistent with DataFrame behavior). To access a value by position, u
se `ser.iloc[pos]`
    axs[i-1].annotate(np.array(city_coors.index)[j], (x[j], y[j]))
```



By observing the repartition of the clusters on the french map we can observe that :

- Augmenting the number of clusters bring some interesting specifications. Indeed for 5 clusters cities which are close to the alps have their own cluster and hotter cities are splitted between mediterranean coast cities and cities further to the coast (as Toulouse and Bordeaux).
- But the clusters defined by the 3-clusters kmeans algorithm are fluctuating while the number of clusters augments (ex : Nantes is integrated in the Toulouse's and Bordeaux's group). Also, the number of points being 15, too much clusters could lead to risks of overfitting (indeed it's already over the empirical maximum of clusters equal to $\sqrt{15} = 3.87$).

To conclude raising the number of clusters can lead to more subtle climate differentiation but can also break the more simple and 'evident' groups we have for 3 clusters. Depending on the precise objective of a data scientist a number 3, 4 or 5 clusters can be interesting. We don't go into further numbers of clusters as putting one cluster for two points will sure lead to overfitting

Digits

```
In [ ]: X_digits = digit_data['x']
X_digits
```

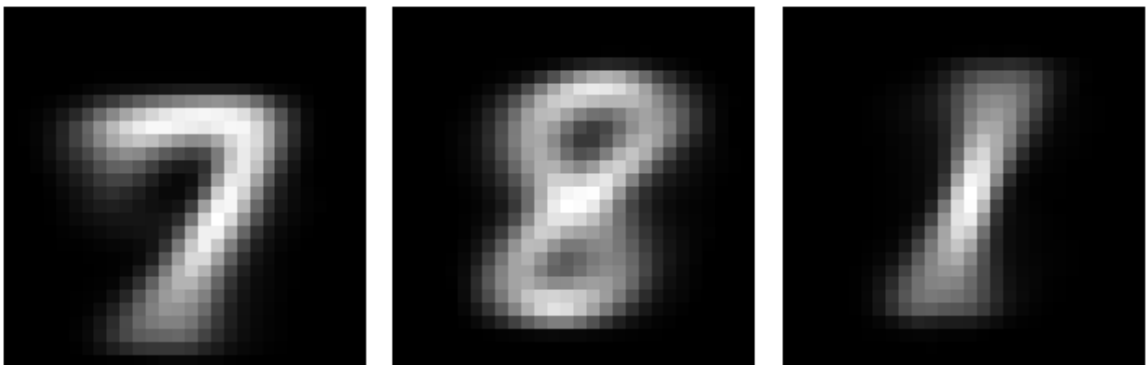
```
Out[ ]: array([[0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               ...,
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0]], shape=(3000, 784), dtype=uint8)
```

```
In [ ]: kmeans_digits = cluster.KMeans(3, random_state=42)
kmeans_digits.fit(X_digits)
```

```
Out[ ]: ▼ KMeans ⓘ ?
        ► Parameters
```

```
In [ ]: centroids_digits = kmeans_digits.cluster_centers_
labels_digits = kmeans_digits.labels_
```

```
In [ ]: display_images(centroids_digits)
```



These centroids seems to correspond to a generic (widely comparable) version of each number represented on the data. Indeed these centroids don't seem to correspond to real data points as they have a bit of blur and are thicker than a random image :

```
In [ ]: display_images([X_digits[500 + 1000*i] for i in range(3)])
```



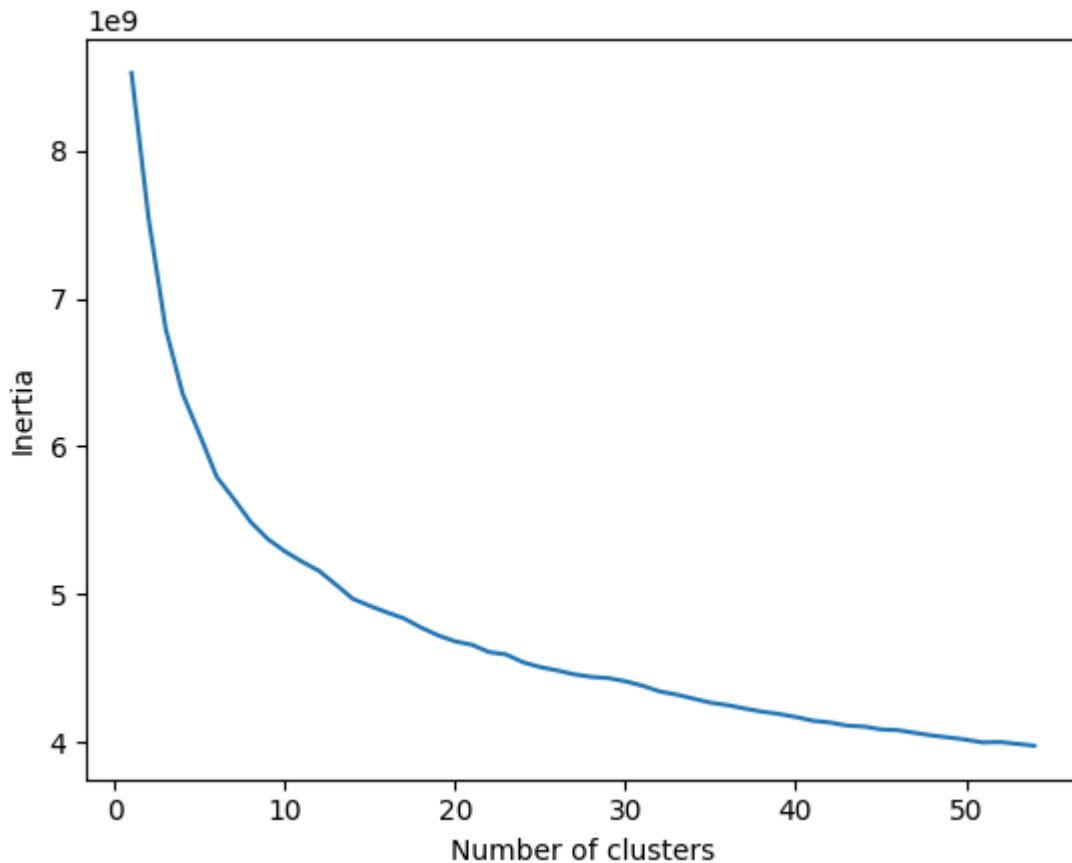
To determinate the adequate number of clusters let's try to use the elbow method (we fix the maximal number of clusters at 55 as $\sqrt{(3000)} = 54.8$)

```
In [ ]: ## Compute and display inertia

Kmax = 55
inertia_digits = []
kmeans = []

for i in range(1, Kmax) :
    kmeans_temp = cluster.KMeans(i, random_state = 42)
    kmeans_temp.fit(X_digits)
    kmeans.append(kmeans_temp)
    inertia_digits.append(kmeans_temp.inertia_)
```

```
In [ ]: plt.figure()
plt.plot(np.arange(1,Kmax), inertia_digits)
plt.xlabel("Number of clusters")
plt.ylabel("Inertia")
plt.show()
```



Here the inertia curve is too smooth to select a proper optimal value of k , let's compute other clustering performance evaluation :

```
In [ ]: ## Compute Silhouette and Davis Bouldin Scores

from sklearn.metrics import davies_bouldin_score
from sklearn.metrics import silhouette_score

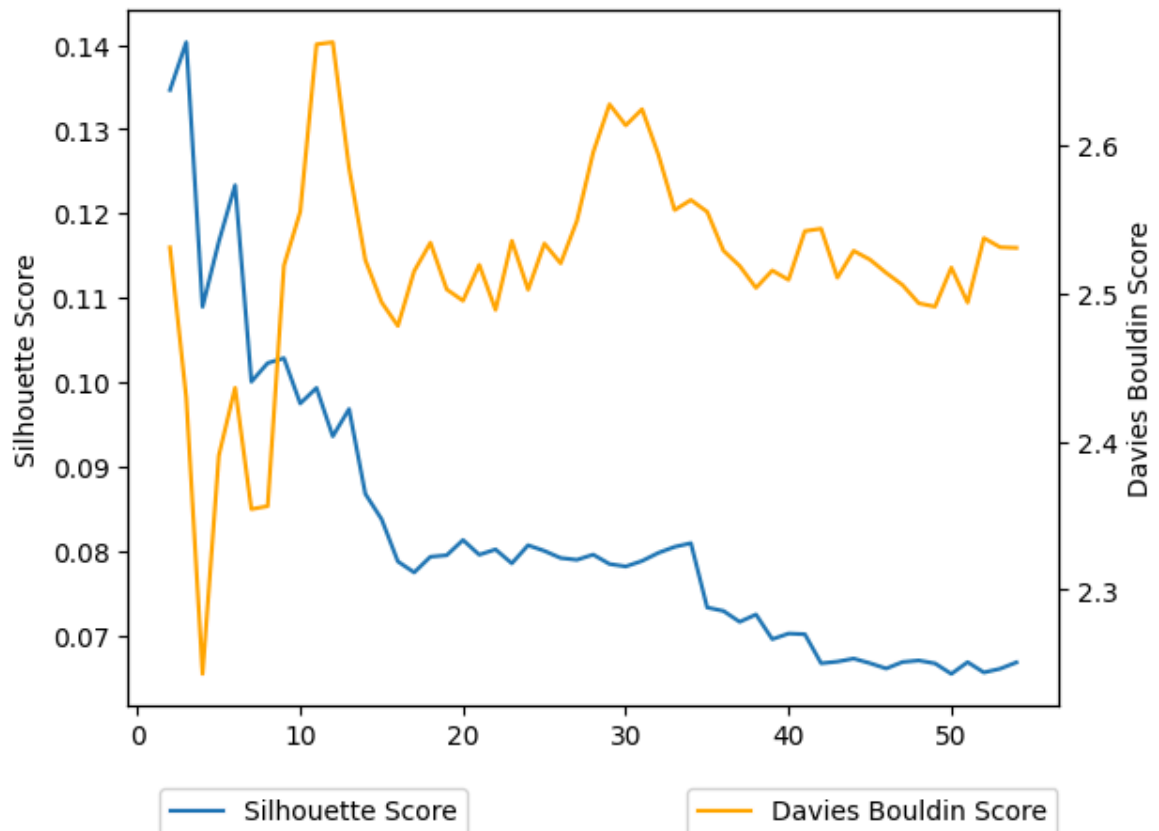
sil = []
dbs = []

for i in range(1, Kmax-1) :
    kmeans_temp = kmeans[i]
    sil.append(silhouette_score(X_digits, kmeans_temp.labels_))
    dbs.append(davies_bouldin_score(X_digits, kmeans_temp.labels_))
```

```
In [ ]: fig, ax1 = plt.subplots()

ax1.plot(np.arange(2,Kmax), sil, label = "Silhouette Score")
ax1.set_ylabel("Silhouette Score")
ax1.legend(loc='upper center', bbox_to_anchor=(0.2, -0.1))
ax2 = ax1.twinx()
ax2.set_ylabel("Davies Bouldin Score")
ax2.plot(np.arange(2,Kmax), dbs, color = 'orange', label = "Davies Bouldi
ax2.legend(loc='upper center', bbox_to_anchor=(0.8, -0.1))

plt.xlabel("Number of clusters")
plt.show()
```



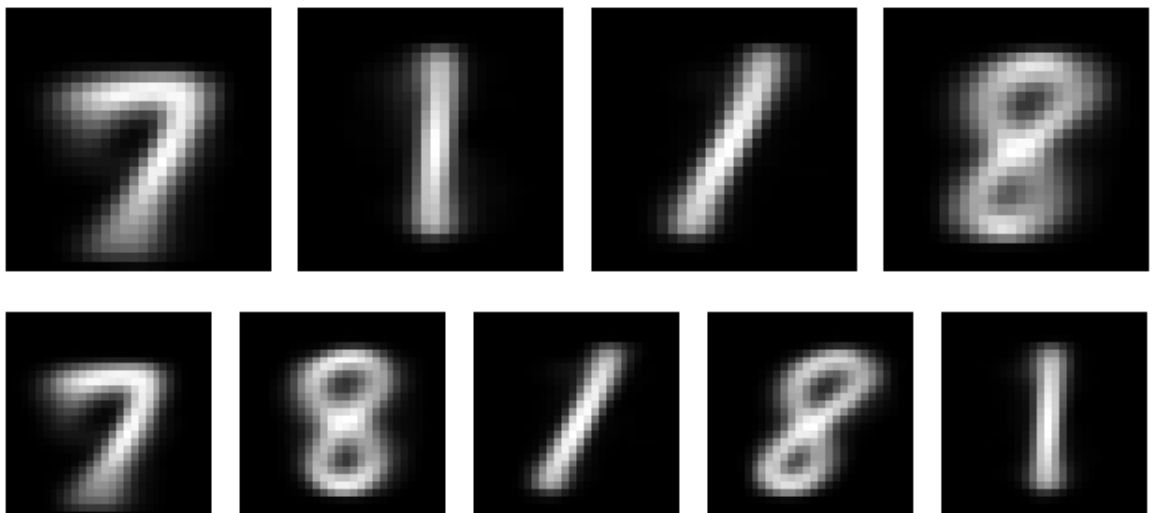
```
In [ ]: nb_opt_sil = np.argmax(sil) + 2
        nb_opt_dbs = np.argmin(dbs) + 2
        nb_opt_sil, nb_opt_dbs
```

```
Out[ ]: (np.int64(3), np.int64(4))
```

- According to the Silhouette Score, the best number of clusters is 3
- According to the Davies Boudlin Score it is 4

Let's visualize the centroids of kmeans clustering for 4 and 5 clusters :

```
In [ ]: display_images(kmeans[3].cluster_centers_)
        display_images(kmeans[4].cluster_centers_)
```



We can observe that the supplementary clusters created allow to distinguish different typos for a number. As we want to differentiate the number augmenting the number of clusters don't seem pertinent : we keep a value of $k = 3$.

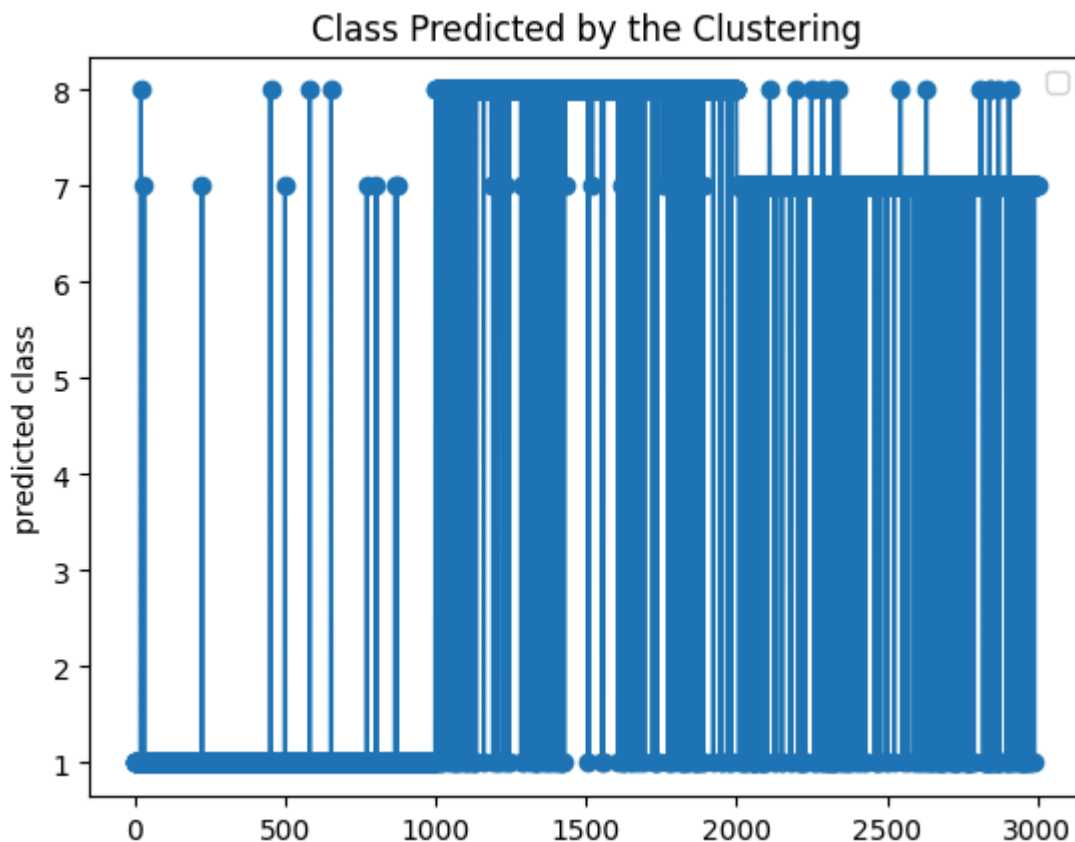
Now we associate each cluster to the class it obviously correspond

```
In [ ]: ## Replace labels by numbers represented by the clusters
def labels_translation(labels_digits) :
    return [1 if x == 2 else 8 if x == 0 else 7 for x in labels_digits]
labels_digits_t = labels_translation(labels_digits)
labels_digits_t[:10]
```

```
Out[ ]: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

```
In [ ]: plt.figure()
plt.plot(np.arange(0, len(labels_digits_t)), labels_digits_t, marker = 'o')
plt.ylabel("predicted class")
plt.title("Class Predicted by the Clustering")
plt.legend()
plt.show()
```

/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/2997290551.py:5: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend()



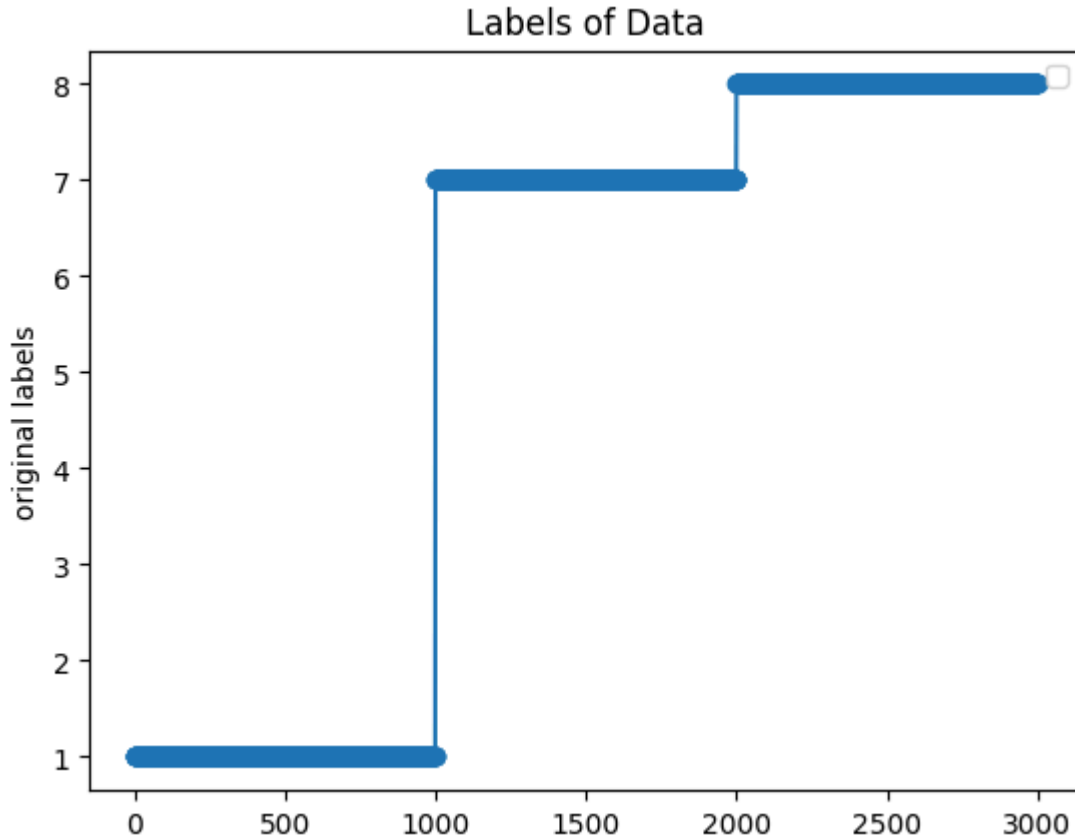
```
In [ ]: labels_digits_GT = digit_data['y']

plt.figure()
plt.plot(np.arange(0, len(labels_digits_t)), labels_digits_GT, marker = 'o')
plt.ylabel("original labels")
```



```
plt.title("Labels of Data")
plt.legend()
plt.show()
```

```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/157061584
2.py:7: UserWarning: No artists with labels found to put in legend. Note
that artists whose label start with an underscore are ignored when legend
() is called with no argument.
  plt.legend()
```



As the ground truth labels function is piecewise constant we can conclude that errors persists in the clustering of digits data

```
In [ ]: display_images(X_digits[20:35])
display_images(X_digits[1035:1050])
```



As the typos for a number can vary a lot, it is possible that k-means can't perfectly separate the data. Especially the 1 can look like a 7 and inversely.

```
In [ ]: from sklearn import metrics
```

```
In [ ]: labels_digits_GT = np.transpose(digit_data['y'])[0]
cluster_perf = {3 : [], 4 : [], 5 : [], 10 : []}
```

```
In [ ]: ## Random Score
```

```

rand_score_digits = metrics.rand_score
rs = rand_score_digits(labels_digits_GT, np.transpose(labels_digits))
cluster_perf[3].append(rs)
rs

```

Out[]: 0.8851168167166833

```

In [ ]: ## Adjusted Random Score

adjusted_rand_score_digits = metrics.adjusted_rand_score
ars = adjusted_rand_score_digits(labels_digits_GT, labels_digits)
cluster_perf[3].append(ars)
ars

```

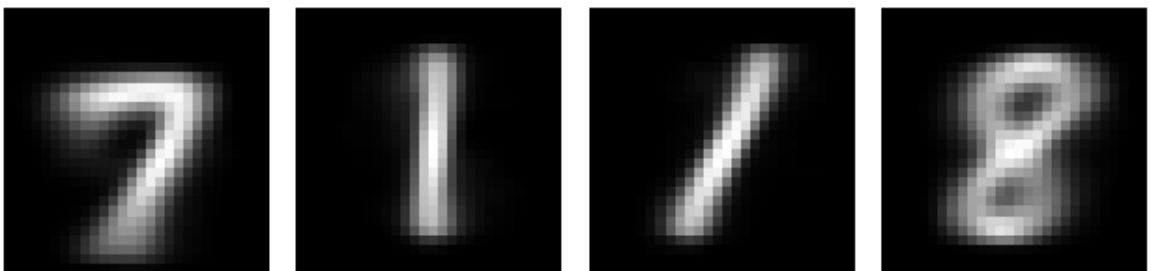
Out[]: 0.7431111778078265

Let's now compare with the scores for different values of K. As it is aberrant to compare labels with different categories, we will regroup clusters by the number they obviously represent and labelize them.

```

In [ ]: display_images(kmeans[3].cluster_centers_)

```



```

In [ ]: ## Transform labels for k = 4

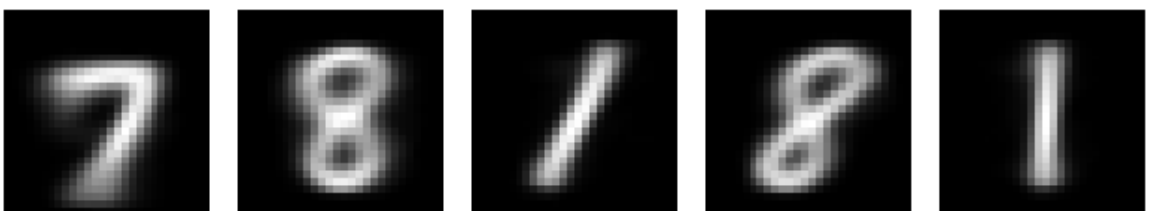
labels_grouped = {}
labels_grouped[4] = [7 if i == 0 else 1 if i in [1,2] else 8 for i in kme

```

```

In [ ]: display_images(kmeans[4].cluster_centers_)

```



```

In [ ]: ## Transform labels for k = 5

labels_grouped[5] = [7 if i == 0 else 1 if i in [2,4] else 8 for i in kme

```

```

In [ ]: display_images(kmeans[9].cluster_centers_)

```



```
In [ ]: ## Transform labels for k = 10

labels_grouped[10] = [7 if i in [0,4, 6, 9] else 1 if i in [1,2,3] else 8
```

```
In [ ]: ## Compute performances

idx = list(cluster_perf.keys())[1:]
for i in idx :
    rs = rand_score_digits(labels_digits_GT, labels_grouped[i])
    cluster_perf[i].append(rs)
    ars = adjusted_rand_score_digits(labels_digits_GT, labels_grouped[i])
    cluster_perf[i].append(ars)
```

```
In [ ]: cluster_perf
```

```
Out[ ]: {3: [0.8851168167166833, 0.7431111778078265],
         4: [0.8979328665110592, 0.7714965890902817],
         5: [0.9236541069245304, 0.8285424809820144],
         10: [0.9543403356674447, 0.8972823089241884]}
```

```
In [ ]: pd.DataFrame(cluster_perf, index=['random_score', 'adjusted_random_score'])
```

```
Out[ ]:
```

	3	4	5	10
random_score	0.885117	0.897933	0.923654	0.954340
adjusted_random_score	0.743111	0.771497	0.828542	0.897282

Here are the results, we can observe that by observing k and differentiating different typos for each number we can significantly raise the score of the prediction. But it could not be applicable in real cases as the regrouping has to be done manually (or using a more performant predictor but then it would be better to use it).

Density Estimation

```
In [ ]: from sklearn.mixture import GaussianMixture
```

Temperature

With 2 gaussians :

```
In [ ]: Nb_gaussian = 2
```

```
In [ ]: gm_temper = GaussianMixture(Nb_gaussian, covariance_type='diag')
gm_temper.fit(temper_data)
```

```
Out[ ]:
```

▼ GaussianMixture ⓘ ⓘ
 ► Parameters

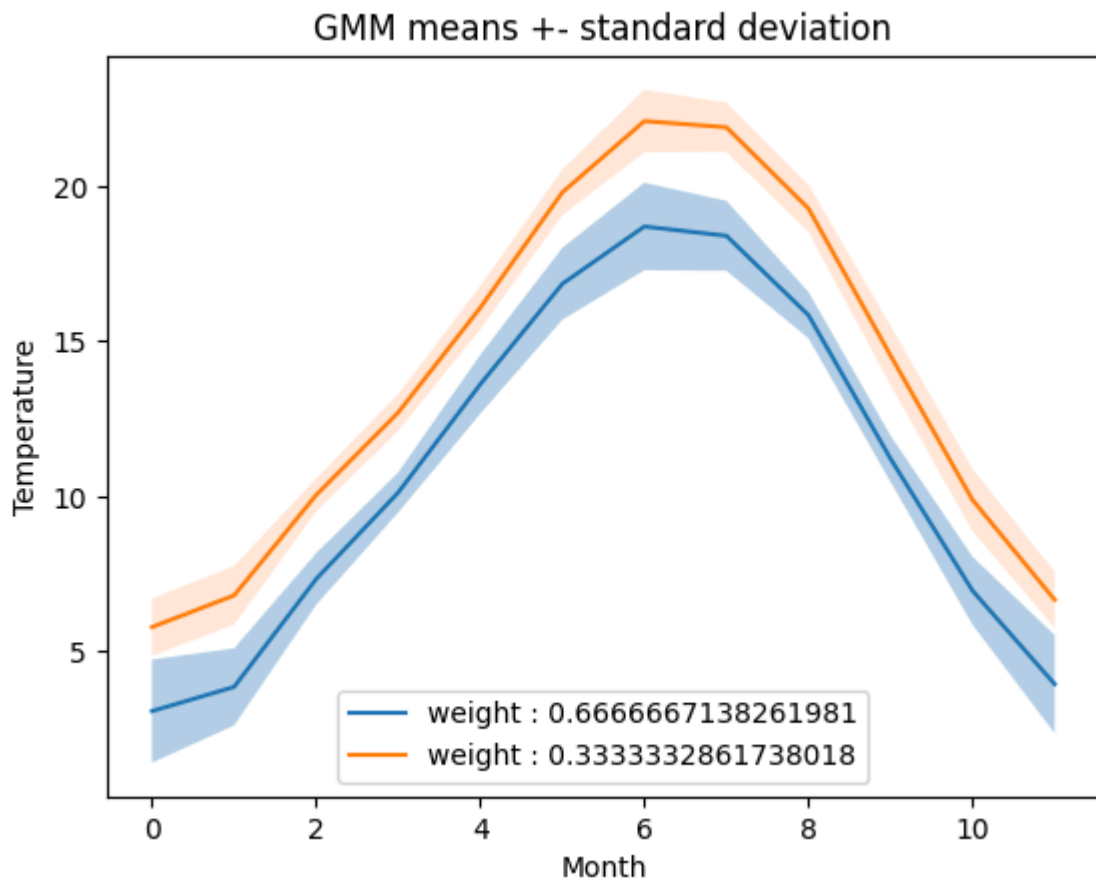
```

In [ ]: means_temper = gm_temper.means_
weights_temper = gm_temper.weights_
covs_temper = gm_temper.covariances_

In [ ]: def display_profiles_GMM(gm_temper, Nb_gaussian) :
    means_temper = gm_temper.means_
    weights_temper = gm_temper.weights_
    covs_temper = gm_temper.covariances_
    display_profiles(means_temper, labels = ["weight : " + str(w) for w in
    for i in range(Nb_gaussian) :
        if gm_temper.covariance_type == 'diag' or gm_temper.covariance_ty
            std = np.sqrt(covs_temper[i])
        else :
            std = np.sqrt(covs_temper[i][i])
        plt.fill_between(temper_data.columns, means_temper[i] - std, mean
    plt.legend()

In [ ]: plt.figure()
display_profiles_GMM(gm_temper, Nb_gaussian)
plt.title("GMM means +- standard deviation")
plt.show()

```



Here as in kmeans the gaussians seems to represent two principal groups of points : the southern/hot cities and the center-northern/cold cities. Also the weight of the gaussians seems to correspond to the number of cities within each group.

```

In [ ]: distrib_temper = np.exp(gm_temper.score_samples(temper_data))
distrib_temper

```

```
Out [ ]: array([3.98886927e-06, 4.82912689e-14, 2.90512407e-06, 1.04960975e-07,
                1.10279391e-09, 9.77786764e-09, 2.21330841e-06, 4.87014345e-05,
                3.36193777e-08, 1.62114398e-09, 1.99266980e-06, 3.10059395e-07,
                4.25403251e-11, 4.64161630e-10, 3.26798800e-06])
```

With other numbers of clusters

```
In [ ]: def generate_GMMs(Nb_gaussian_test, cov_type) :
        gm_temperers = []
        for nb_gaussian in Nb_gaussian_test :
            gm_temper_temp = GaussianMixture(nb_gaussian, covariance_type=cov_type)
            gm_temper_temp.fit(temper_data)
            gm_temperers.append(gm_temper_temp)
        return gm_temperers
```

```
In [ ]: Nb_gaussians_test = [2, 3, 4, 5, 7, 10]

diag_gmms = generate_GMMs(Nb_gaussians_test, 'diag')
```

```
In [ ]: def display_gmms(gmms, Nb_gaussians_test, cols = temper_data.columns, size = (10, 10)):
        fig, axs = plt.subplots(size[0], size[1], figsize = (20, size[0]*6))

        for i in range(len(Nb_gaussians_test)):

            gm_temp = gmms[i]
            means_temper = gm_temp.means_
            weights_temper = gm_temp.weights_
            covs_temper = gm_temp.covariances_

            for j in range(len(means_temper)) :

                if size[0] > 1 :
                    a = axs[i//3, i%3]
                else :
                    a = axs[i]

                ## Plot Mean of Gaussian
                a.plot(cols, means_temper[j], label = "weight : " + str(weights_temper[j]))

                ## Plot Standard Deviation Interval
                if gm_temp.covariance_type == 'diag' or gm_temp.covariance_type == 'full':
                    std = np.sqrt(covs_temper[j])
                else :
                    std = np.sqrt(covs_temper[j][j])
                a.fill_between(temper_data.columns, means_temper[j] - std, means_temper[j] + std, color = 'red', alpha = weights_temper[j])

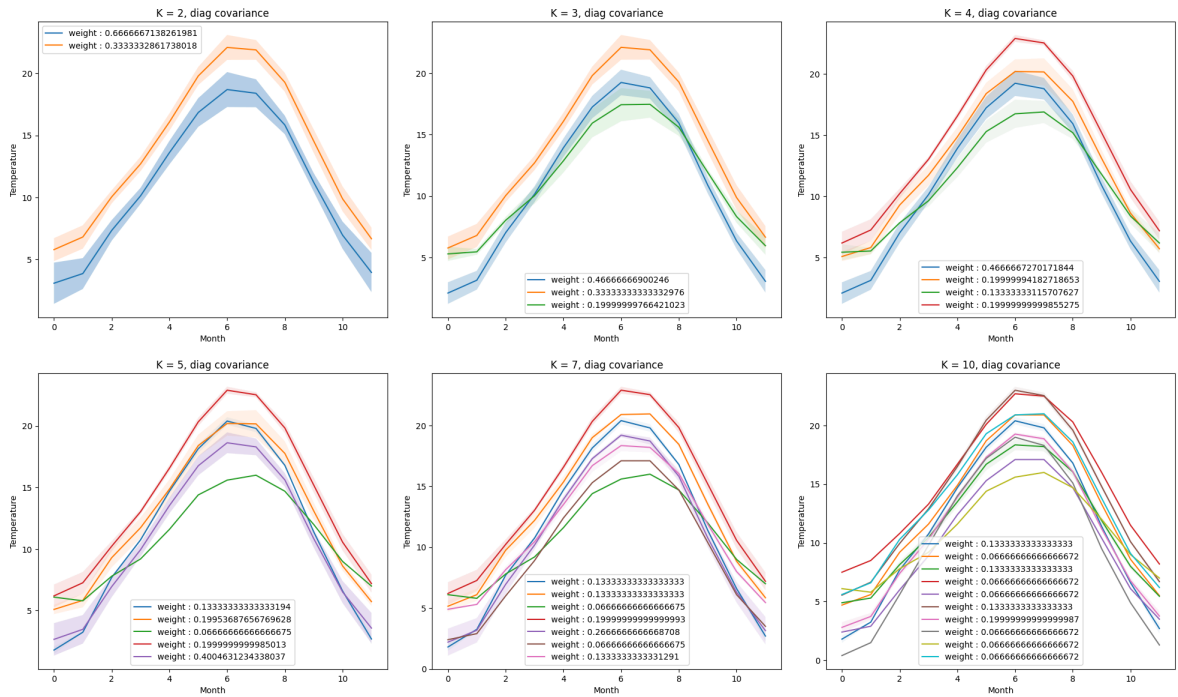
            a.set_ylabel("Temperature")
            a.set_xlabel("Month")
            a.set_title(f"K = {Nb_gaussians_test[i]}, {gm_temp.covariance_type}")
            a.legend()

        plt.tight_layout(pad=2.0)
        plt.suptitle("GMM Means +- Standard Deviation (Opacity ~ Weight) for Nb_gaussians_test")
```

```
In [ ]: display_gmms(diag_gmms, Nb_gaussians_test)
```

```
plt.show()
```

GMM Means +- Standard Deviation (Opacity ~ Weight) for Different Values of K



From a number of 5 gaussian distributions, some of them seems negligible (since their weight is less than 10% or their center/ standard deviation seems to be redundant). Since then we consider the best number of gaussians is 4

Now let's try for different forms of covariance matrices

```
In [ ]: Nb_gaussians_covs = [4, 4, 4]

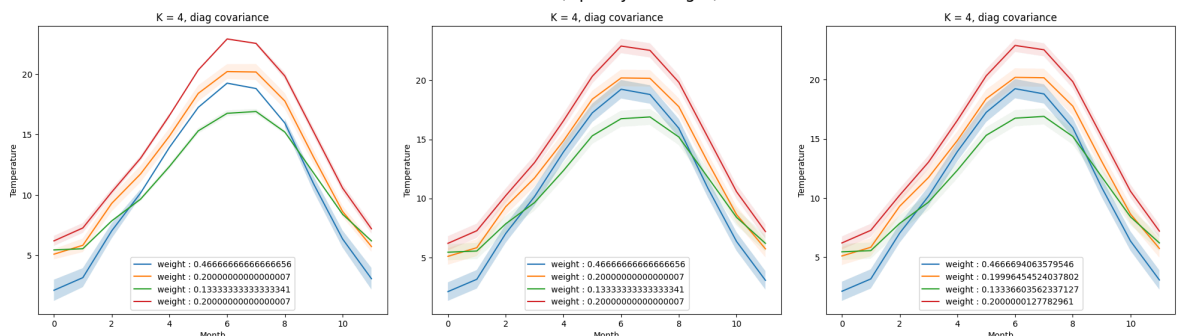
n = [Nb_gaussians_covs[0]]

full_gmms = generate_GMMs(n, 'full')
tied_gmms = generate_GMMs(n, 'tied')
shperical_gmms = generate_GMMs(n, 'spherical')
gmms_covs = np.concatenate([full_gmms, tied_gmms, shperical_gmms])

display_gmms(gmms_covs, Nb_gaussians_covs, size = [1,3])
plt.show()
```

```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/391686166
2.py:26: RuntimeWarning: invalid value encountered in sqrt
std = np.sqrt(covs_temper[j][j])
```

GMM Means +- Standard Deviation (Opacity ~ Weight) for Different Values of K



Since the covariance type don't change much the gaussian distributions and their weights we choose the simplest and less demanding in computation costs : the diagonal covariance matrix

```
In [ ]: temper_gmm = diag_gmms[2]
```

```
In [ ]: temper_scores = temper_gmm.score_samples(temper_data)
temper_scores
```

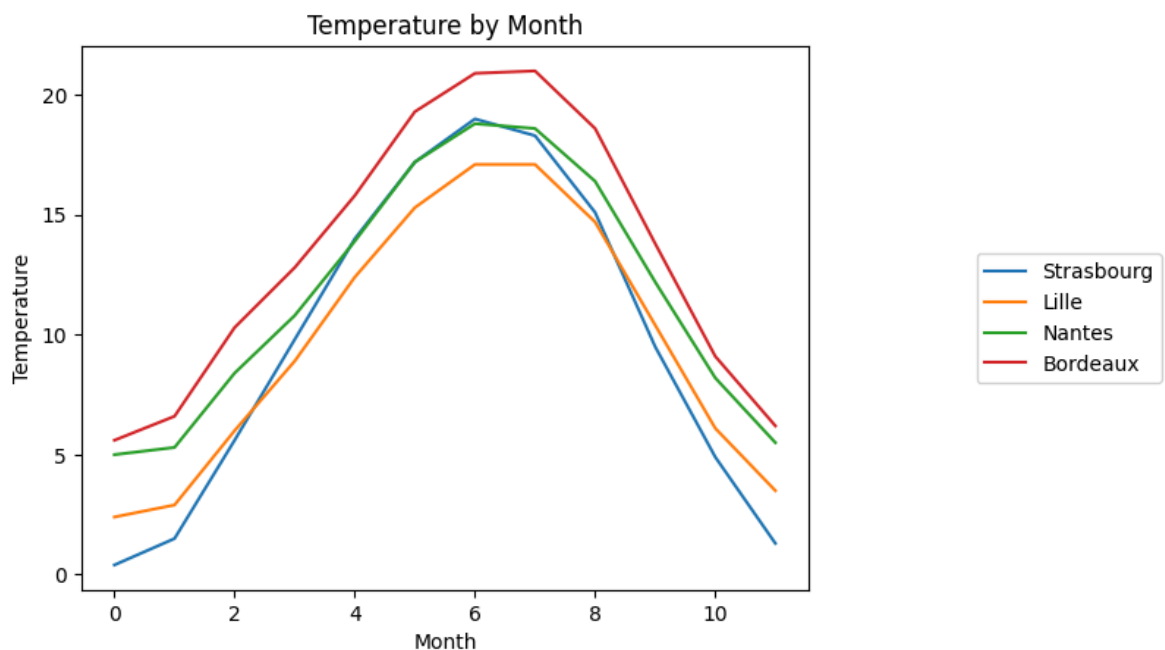
```
Out [ ]: array([-15.80161415, -10.0356526 , -10.09286192, -12.17301471,
               -22.16423484, -15.19958636, -6.79424244, -8.20735739,
               -16.13786758, -10.45876658, -13.48102508, -10.03565296,
               -22.45816141, -9.45546145, -9.45209303])
```

With 4 gaussians

Let's select the least probable indexes and then display the associated curve and city to see if they may be outliers

```
In [ ]: Nb_low_proba = 4
```

```
In [ ]: idx_low = np.argsort(temper_scores)[:Nb_low_proba]
display_profiles([temper_data.iloc[i] for i in idx_low], labels=list(temp
```



These curves corresponds to real cities and don't seem to contain aberrant values : they're not outliers

```
In [ ]: temper_data.iloc[idx_low].index
```

```
Out [ ]: Index(['Strasbourg', 'Lille', 'Nantes', 'Bordeaux'], dtype='object')
```

Digits Data

```
In [ ]: X_digits
```

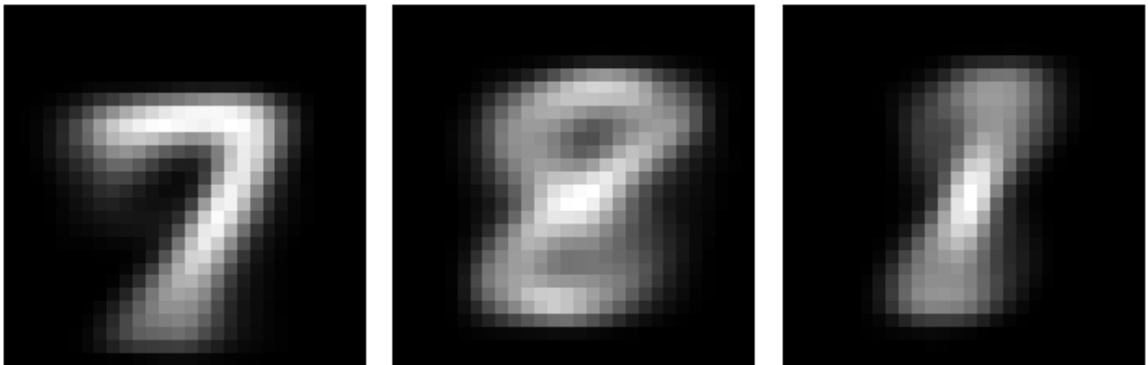
```
Out[ ]: array([[0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               ...,
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0]], shape=(3000, 784), dtype=uint8)
```

```
In [ ]: Nb_gaussians_digits = [3, 5, 10]
```

```
In [ ]: gmm_digits = []
        for nb in Nb_gaussians_digits :
            gm_temp = GaussianMixture(nb, covariance_type= 'diag', random_state=4)
            gm_temp.fit(X_digits)
            gmm_digits.append(gm_temp)
```

With 3 gaussians

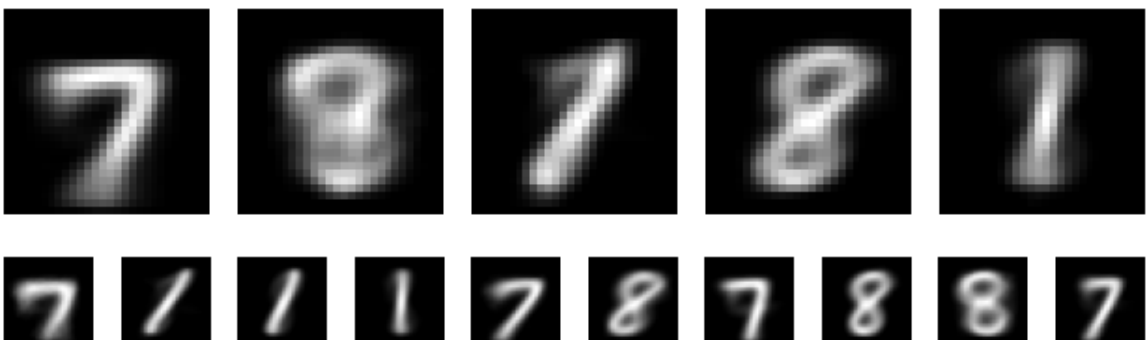
```
In [ ]: display_images(gmm_digits[0].means_)
```



As for the kmeans centroids, the means of estimated gaussians are a blur and thick version of the numbers (so the gaussian distribution encapsulate better all the typos for each number)

With other numbers of gaussians

```
In [ ]: display_images(gmm_digits[1].means_)
        display_images(gmm_digits[2].means_)
```



As in the clustering part for more than 3 clusters, the centers seems redundant. This time some centers seems to be a mix between 1 and 7, maybe there is some data group for which there is confusion, let's check the weights.

```
In [ ]: print("Weights for k = 5 : ", gmm_digits[1].weights_)
        print("-----")
        print("Weights for k = 10 : ", gmm_digits[2].weights_)

Weights for k = 5 : [0.198      0.10166667 0.17733333 0.19466667 0.328333
33]
-----
Weights for k = 10 : [0.04100001 0.06      0.1625262  0.18180713 0.06166
667 0.09833378
0.12233329 0.121      0.05833333 0.09299959]
```

- For the GMM-10, the gaussian looking like a mix of 1 and 7 (the 2nd) has a very small (almost negligible) weight.
- For the GMM-5, the 3rd gaussian has an important weights and the number it is representing is not clear at all. So for a clustering it could lead to interpretability problems, for example we couldn't use the same procedure as for kmeans (grouping clusters for $k > 3$).

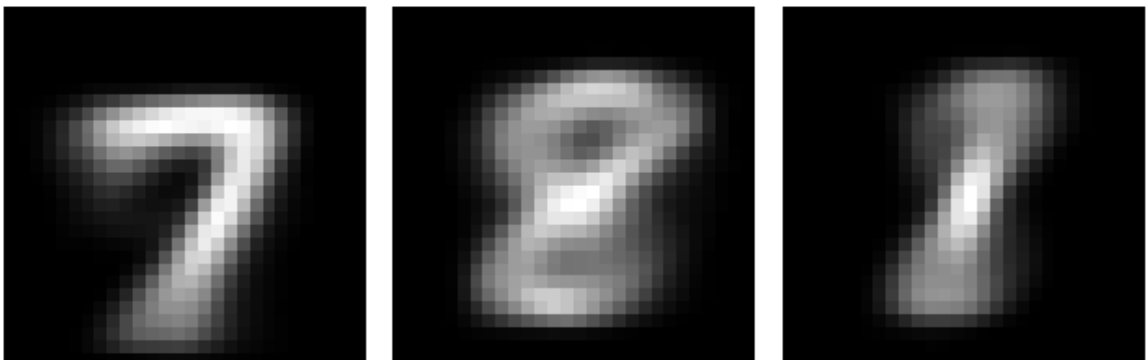
We keep the GMM-3 for the rest of this part, as each gaussian distribution corresponds to one of the three numbers represented within the data

Other covariance types

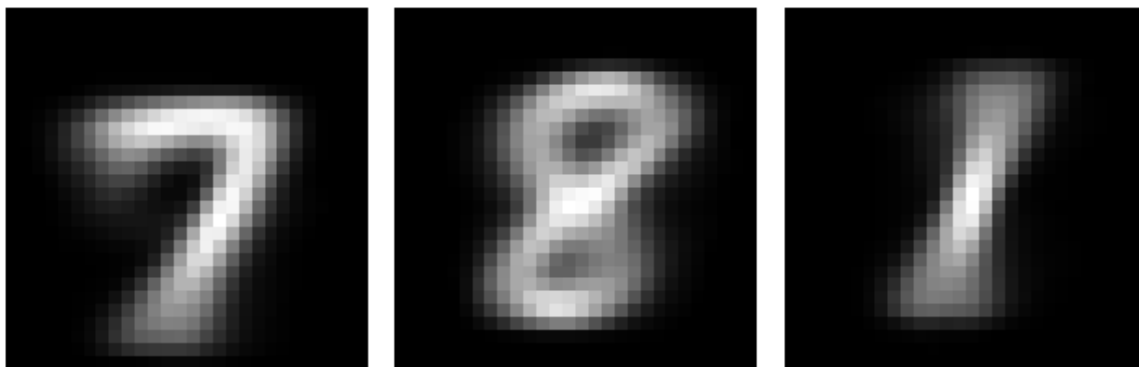
```
In [ ]: gmm_digits_covs = []
        types_covs = ['full', 'tied', 'spherical']
        gmm_digits_covs.append(gmm_digits[0])
        for type_cov in types_covs :
            gm_temp = GaussianMixture(3, covariance_type=type_cov, random_state=4)
            gm_temp.fit(X_digits)
            gmm_digits_covs.append(gm_temp)
        types_covs = ['diag'] + types_covs
```

```
In [ ]: for i in range(len(gmm_digits_covs)) :
        print("Gaussians Centers for Covariance Type : " + types_covs[i])
        display_images(gmm_digits_covs[i].means_)
```

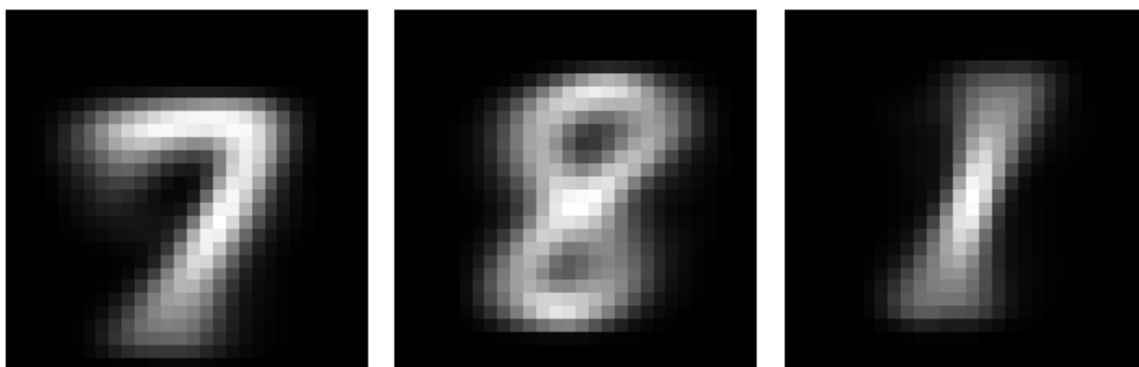
Gaussians Centers for Covariance Type : diag



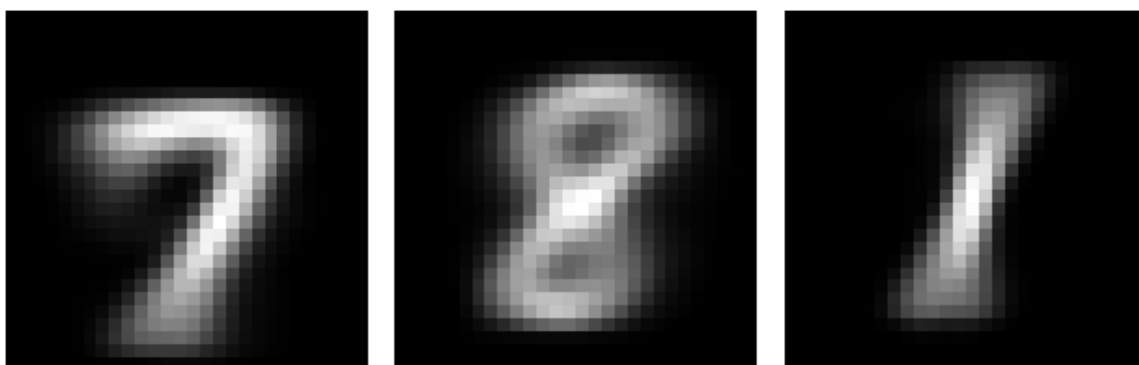
Gaussians Centers for Covariance Type : full



Gaussians Centers for Covariance Type : tied



Gaussians Centers for Covariance Type : spherical



Results are exactly the same. With other types of covariance than 'diag' (especially for 'full'), computation times are way longer. So we use diag which is the simplest representative allowing different variances depending on the number concerned ('tied' not so good in this case)

Outlier detection

```
In [ ]: gm_digits = gmm_digits[0]
gm_digits
```

```
Out [ ]: GaussianMixture ⓘ ?
          Parameters
```

```
In [ ]: probas_digits = gm_digits.score_samples(X_digits.astype(np.float64))
```

Let's display the samples with the lowest probability

```
In [ ]: nb_samples = 15

        idx_low_d = np.argsort(probas_digits)[:nb_samples]
        display_images(X_digits[idx_low_d])
```



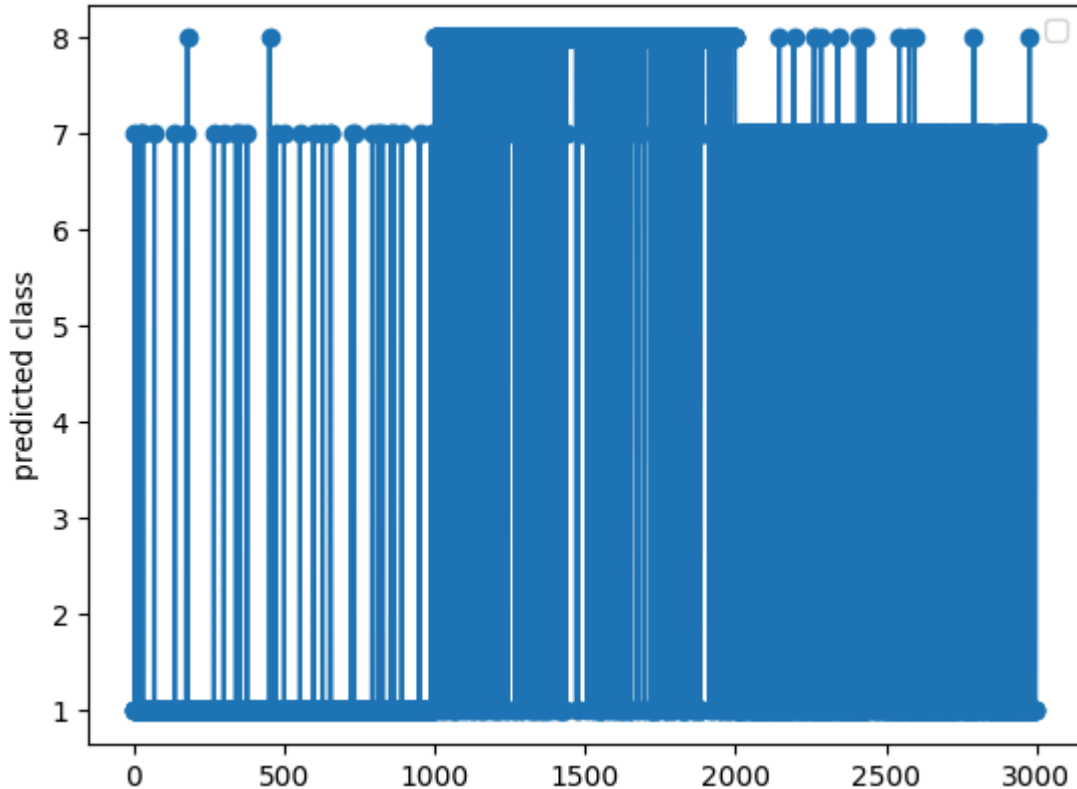
Some of them can be considered as outliers as their typo can lead to confusion on the number represented, even from a human point of view

Now let's cluster the data using our GMM-3

```
In [ ]: gmm_clusters = gm_digits.predict(X_digits.astype(np.float64))
        gmm_clusters = labels_translation(gmm_clusters)
```

```
In [ ]: plt.figure()
        plt.plot(np.arange(0, len(gmm_clusters)), gmm_clusters, marker = 'o')
        plt.ylabel("predicted class")
        plt.legend()
        plt.show()
```

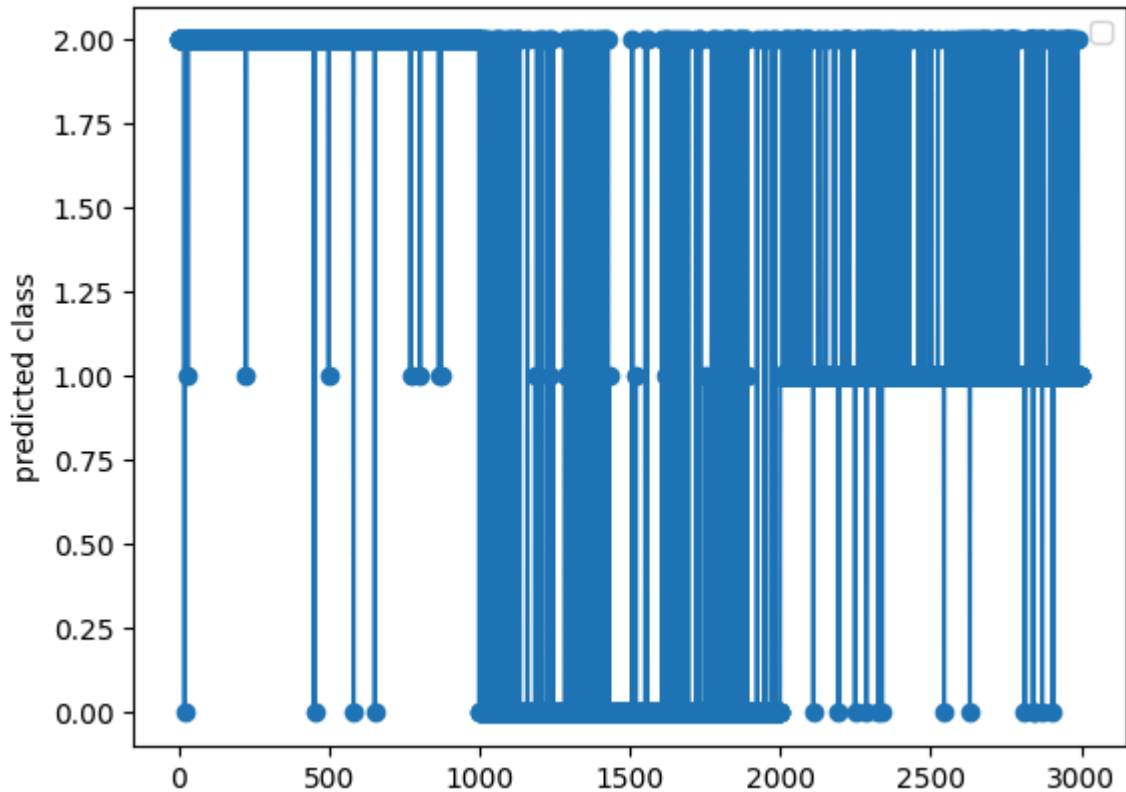
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/2122352298.py:4: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend()



```
In [ ]: plt.figure()
        plt.plot(np.arange(0, len(labels_digits)), labels_digits, marker = 'o')
        plt.ylabel("predicted class")
```

```
plt.legend()
plt.show()
```

```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/26531756
9.py:4: UserWarning: No artists with labels found to put in legend. Note
that artists whose label start with an underscore are ignored when legend
() is called with no argument.
plt.legend()
```



At first look the GMM clustering seems to lead to less accuracy regarding data's labels. Here k is fixed : $k = 3$

```
In [ ]: scores = {}
rand_score_digits = metrics.rand_score
adjusted_rand_score_digits = metrics.adjusted_rand_score

s_temp = []
s_temp.append(rand_score_digits(labels_digits_GT, labels_digits))
s_temp.append(adjusted_rand_score_digits(labels_digits_GT, labels_digits))
scores['kmeans'] = s_temp

for i in range(len(gmm_digits_covs)):
    clusters_temp = gmm_digits_covs[i].predict(X_digits.astype(np.float64))
    s_temp = []
    s_temp.append(rand_score_digits(labels_digits_GT, clusters_temp))
    s_temp.append(adjusted_rand_score_digits(labels_digits_GT, clusters_t
    scores[f'gmm ({gmm_digits_covs[i].covariance_type} covariance)'] = s_

res = pd.DataFrame(scores, index = ['random score', 'adjusted random scor
res
```

Out []:

	kmeans	gmm (diag covariance)	gmm (full covariance)	gmm (tied covariance)	gmm (spherical covariance)
random score	0.885117	0.740756	0.863577	0.898686	0.903653
adjusted random score	0.743111	0.441323	0.696216	0.772979	0.783577

We can observe that depending on the type of covariance choosed the results can be different.

- For diagonal and Full covariance GMM is less performant than kmeans for clustering.
- For tied and spherical covariance GMM is better than kmeans.

These results can be explained by the fact that the Gaussian Mixture using a multi dimensional covariance matrix while the kmeans only relies on the mean of created groups, especially since the data has quite a high dimensionality.

We can guess that a purely diagonal matrix is too far from the reality in matters of differences between typos for a number, but it is curious that the full covariance leads to less good results than the kmeans (and even than the spherical and tied covariance that have more constraints).

Indeed the random score can't really detect overfitting as it only compares how the clusters fit the fitting data. Maybe this strange results comes from the parameters of the random score or of the optimization of the covariance matrix.

In []: `## Data Generation`

```
X_digits_gmm = gm_digits.sample(45)
display_images(X_digits_gmm[0])
```



They don't really look like real numbers : white points less concentrated --> variance too high maybe

Dimensionality Reduction

```
In [ ]: from sklearn.decomposition import PCA
        from sklearn.manifold import TSNE
```

Temperature Data

Principal Component Analysis

```
In [ ]: temper_data.shape
```

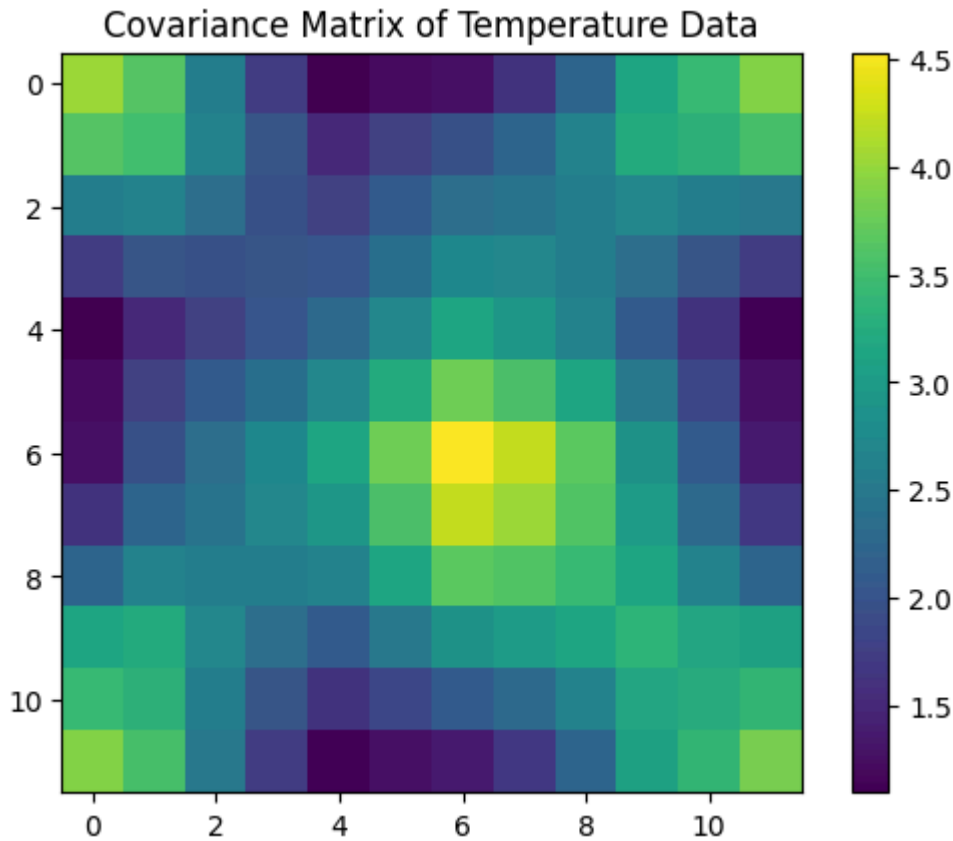
```
Out[ ]: (15, 12)
```

```
In [ ]: cov_temp = np.cov(temper_data.T)
        cov_temp.shape
```

```
Out[ ]: (12, 12)
```

```
In [ ]: ## Define a function to plot covariance matrices as an image :
        def display_cov(cov_matrix, data_title) :
            plt.imshow(cov_matrix)
            plt.title("Covariance Matrix of " + data_title)
            plt.colorbar()
```

```
In [ ]: plt.figure()
        display_cov(cov_temp, "Temperature Data")
        plt.show()
```



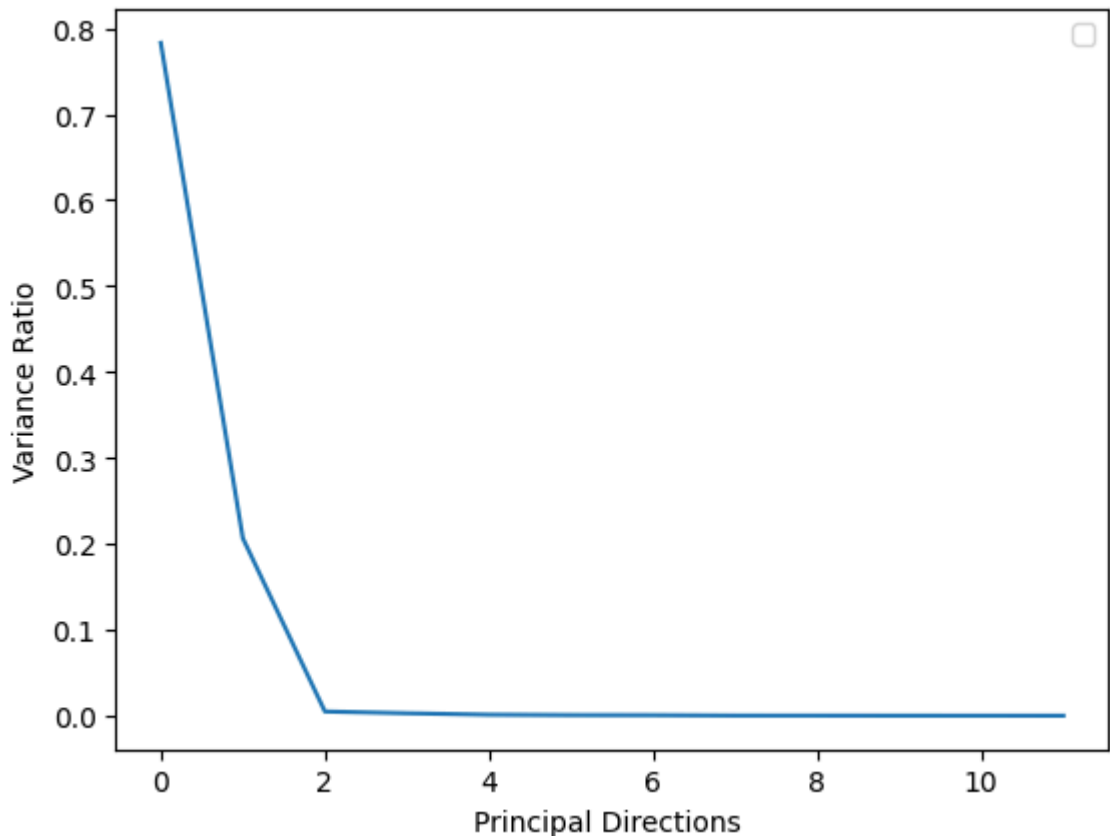
Here the features are the average temperature at each month. We can observe that the temperature at summer and winter months has a stronger variance than the temperature in autumn or spring. Also because the variance increases in these months the covariance between near months is also stronger, and we have strong covariance between months 1 and 12 because the data has a cyclic evolution with period 12.

```
In [ ]: pca_temper = PCA(random_state=42)
pca_temper.fit(temper_data)
```

```
Out [ ]: PCA ⓘ ?
  ► Parameters
```

```
In [ ]: plt.figure()
plt.plot(pca_temper.explained_variance_ratio_)
plt.xlabel("Principal Directions ")
plt.ylabel("Variance Ratio")
plt.legend()
plt.show()
```

```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/219090308
9.py:5: UserWarning: No artists with labels found to put in legend. Note
that artists whose label start with an underscore are ignored when legend
() is called with no argument.
  plt.legend()
```



```
In [ ]: pca_temper.explained_variance_ratio_[2].sum()
```

```
Out[ ]: np.float64(0.9896802035903165)
```

We can consider that except the two first components all of them have a negligible impact, and so we can fix the number of components at 2 (we keep 99% of the information).

```
In [ ]: pca_temper_2 = PCA(2, random_state=42)
transformed_temper_data = pca_temper_2.fit_transform(temper_data)
```

```
In [ ]: temper_villes
```

```
Out[ ]: array(['Bordeaux', 'Brest', 'Clermont-Ferrand', 'Grenoble', 'Lille',
               'Lyon', 'Marseille', 'Montpellier', 'Nantes', 'Nice', 'Paris',
               'Rennes', 'Strasbourg', 'Toulouse', 'Vichy'], dtype='<U16')
```

```
In [ ]: transformed_temper_data[:,1]
```

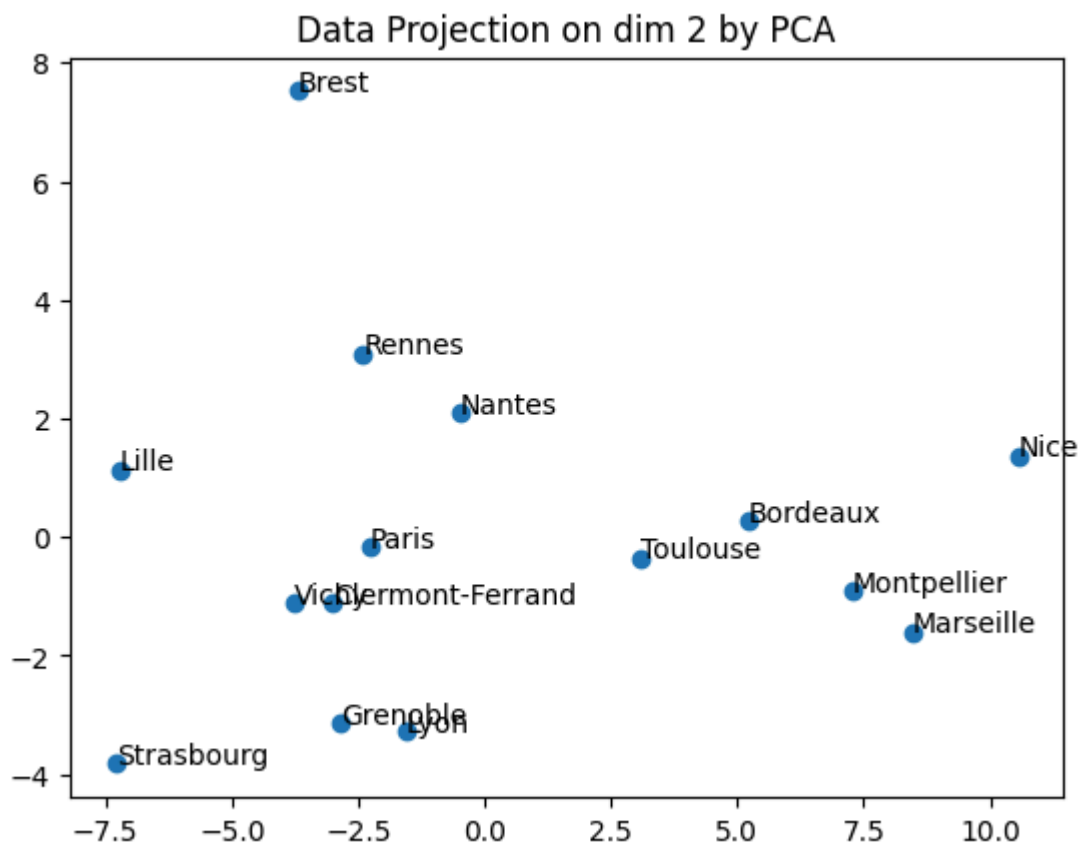
```
Out[ ]: array([ 0.26531184,  7.52111152, -1.11577501, -3.13861729,  1.13343576,
               -3.27964904, -1.60519291, -0.90701735,  2.10198083,  1.36465031,
               -0.15755765,  3.09546059, -3.82428559, -0.35018275, -1.10367325])
```

```
In [ ]: def display_transformed_temper(transformed_temper_data, temper_villes = t
      plt.scatter(transformed_temper_data[:, 0], transformed_temper_data[:,
      for i in range(len(temper_villes)) :
          plt.text(transformed_temper_data[i][0], transformed_temper_data[i
```

```
In [ ]: plt.figure()
display_transformed_temper(transformed_temper_data)
```



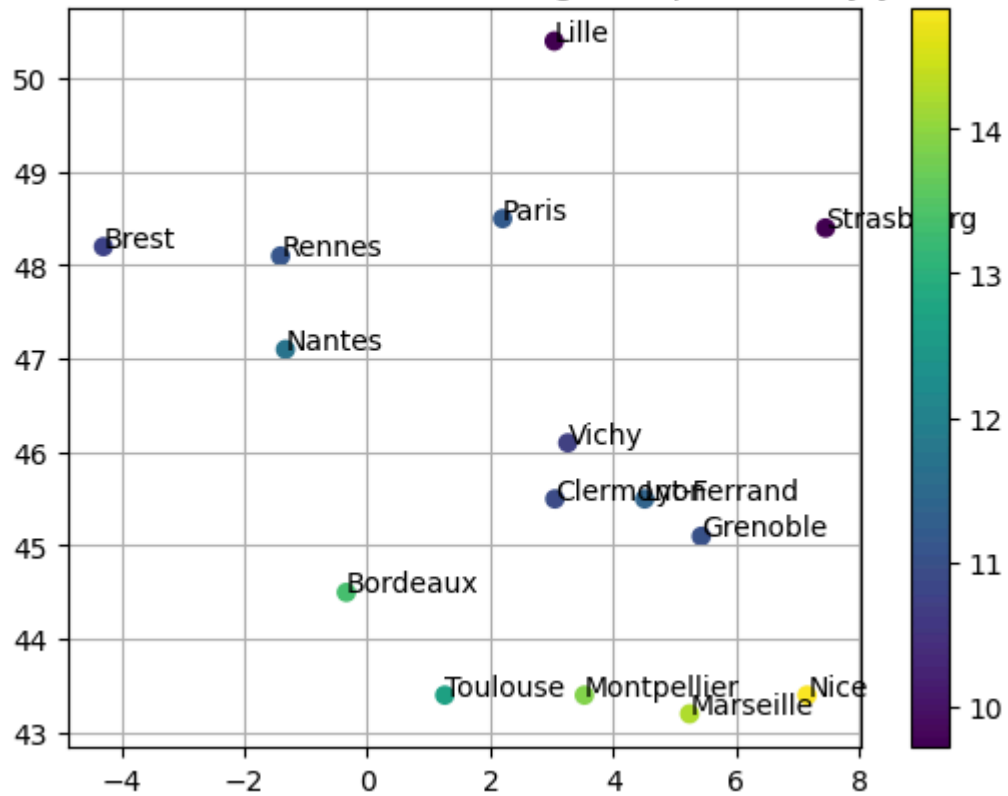
```
plt.title("Data Projection on dim 2 by PCA")
plt.show()
```



```
In [ ]: display_cities(temper_mean)
```

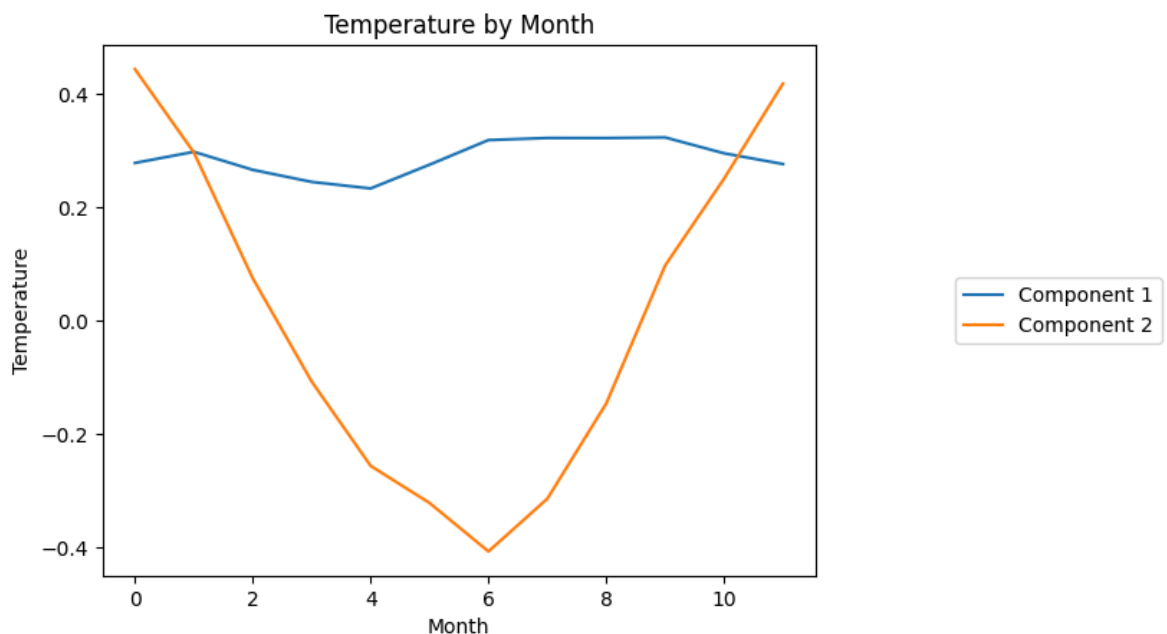
```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/127493545
7.py:8: FutureWarning: Series.__getitem__ treating keys as positions is de
precated. In a future version, integer keys will always be treated as labe
ls (consistent with DataFrame behavior). To access a value by position, us
e `ser.iloc[pos]`
plt.annotate(np.array(city_coords.index)[i], (x[i], y[i]))
```

Position of cities in France + average temperature by year



The 2D PCA representation and the position of cities relatively to others kind of look like their geographical position, although there is some differences.

```
In [ ]: display_profiles(pca_temper_2.components_, labels = ['Component 1', 'Comp
```



- Moving along the 1st component is raising almost in a constant manner the temperature. It can be interpreted as a kind of shift used to get the average temperature of a city (with a slight decrease of temperature around spring and a slight increase in autumn).

- Moving along the 2nd component draws the characteristic form of the curve, higher temperatures in summer and lower temperatures in winter. So for all points the weight used to ponderate this second component is negative.

Now let's reconstitute the data via PCAs of different subspace sizes

```
In [ ]: nb_compos = [2, 3, 4, 5, 9, None]
pcas = []
```

```
In [ ]: for n in nb_compos :
    pca_temp = PCA(n, random_state = 42)
    pca_temp.fit(temper_data)
    pcas.append(pca_temp)
```

```
In [ ]: temper_data_gen_pca = {}

for i in range(len(nb_compos)) :
    pca_temp = pcas[i]
    pca_gen = pca_temp.inverse_transform(pca_temp.transform(temper_data))
    temper_data_gen_pca[nb_compos[i]] = pca_gen
```

```
In [ ]: from cyclor import cyclor
```

```
In [ ]: def display_reconstitution(temper_data_gen_pca, nb_comp, nb_curves = 3, co
fig, axs = plt.subplots(2,3, figsize = (20,12))
    for i in range(len(nb_comp)) :

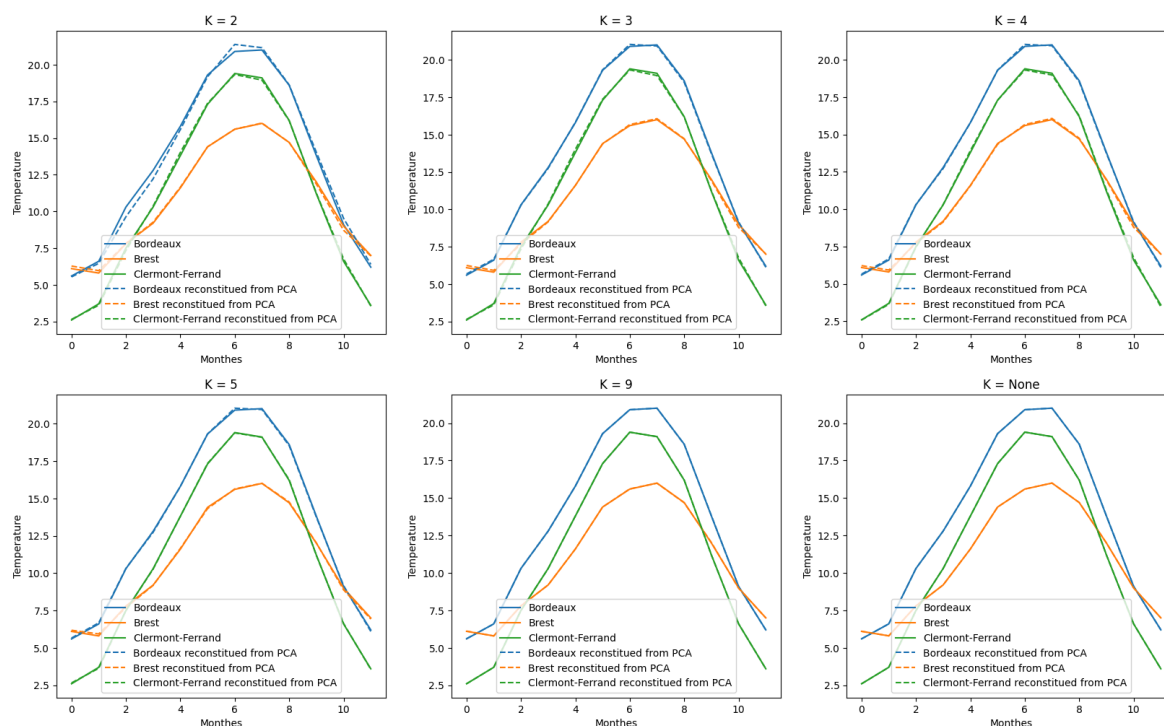
        for j in range(nb_curves) :
            axs[i//3, i%3].plot(np.array(temper_data)[j], label = temper_
            """
            axs[i//3, i%3].set_xlabel("Monthes")
            axs[i//3, i%3].set_ylabel("Temperature")
            axs[i//3, i%3].set_title(f"K = {nb_compos[i]}")
            """

        axs[i//3, i%3].set_prop_cycle(None)
        for j in range(nb_curves) :
            axs[i//3, i%3].plot(temper_data_gen_pca[nb_comp[i]][j], lines
            axs[i//3, i%3].set_xlabel("Monthes")
            axs[i//3, i%3].set_ylabel("Temperature")
            axs[i//3, i%3].set_title(f"K = {nb_comp[i]}")
            axs[i//3, i%3].legend()

    plt.suptitle(f""""Temperature by Month Reconstituted from PCA with Diff
```

```
In [ ]: display_reconstitution(temper_data_gen_pca, nb_compos)
plt.show()
```

Temperature by Month Reconstituted from PCA with Different Values of K



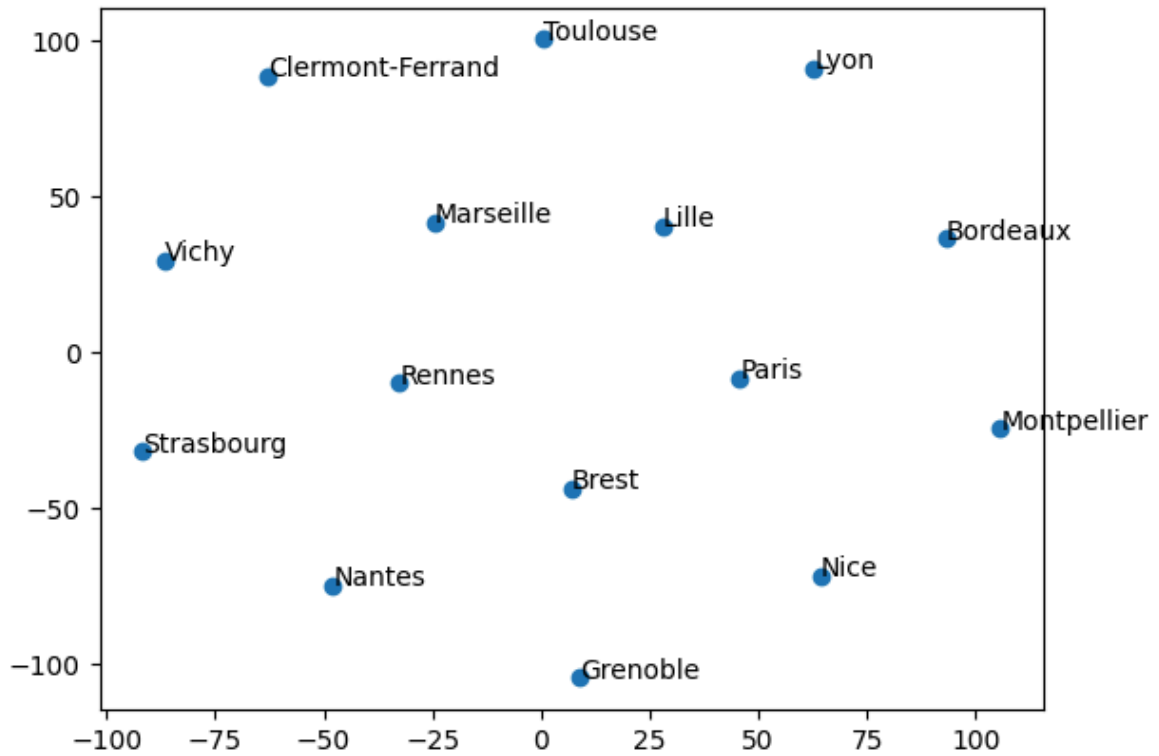
Even if the reconstitution from a 2D PCA can be precise enough, increasing the number of components allows to get perfectly fitted reconstituted curves (from 9 components).

TSNE

```
In [ ]: tsne_temper = TSNE(2, random_state= 42, perplexity= 14.9)
temper_data_tsne = tsne_temper.fit_transform(temper_data)
temper_data_tsne
```

```
Out [ ]: array([[ 93.1462 ,  36.312756 ],
 [  7.001029, -43.78338 ],
 [-62.799877,  88.19337 ],
 [  8.829185, -104.6579 ],
 [ 27.90576 ,  40.065514 ],
 [ 62.866756,  90.92373 ],
 [-24.387554,  41.30884 ],
 [105.6163 , -24.802803 ],
 [-47.96953 , -75.06948 ],
 [ 64.18341 , -71.82015 ],
 [ 45.788857, -8.571444 ],
 [-32.6648 , -10.052712 ],
 [-91.714775, -31.934933 ],
 [  0.5150905, 100.16387 ],
 [-86.72902 ,  29.176273 ]], dtype=float32)
```

```
In [ ]: plt.figure()
display_transformed_temper(temper_data_tsne)
plt.show()
```



These results don't seem to represent the any geographical or temperature schemes.
Let's try for different values of perplexity :

```
In [ ]: perplex = np.linspace(1, 14, 5)

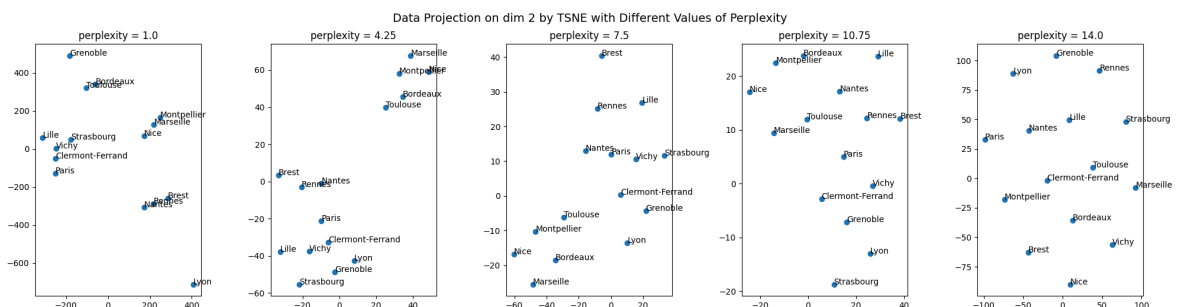
datas_tsne = []

for p in perplex :
    tsne_temper = TSNE(2, random_state= 42, perplexity= p)
    temper_data_tsne = tsne_temper.fit_transform(temper_data)
    datas_tsne.append(temper_data_tsne)

fig, axs = plt.subplots(1,5, figsize = (20,5))

for i in range(len(datas_tsne)) :
    data = datas_tsne[i]
    axs[i].scatter(data[:, 0], data[:, 1])
    axs[i].set_title(f"perplexity = {perplex[i]}")
    for j in range(len(temper_villes)) :
        axs[i].text(data[j][0], data[j][1], temper_villes[j])

plt.tight_layout()
plt.suptitle("Data Projection on dim 2 by TSNE with Different Values of P")
plt.show()
```



- Low perplexity : clear separation by regions. Groups of cities are created, corresponding to geographical groups, but the position of each group relatively to each other don't seem to be geographically representative.
- High perplexity : distance between cities seems to be unified.

Digits Data

PCA

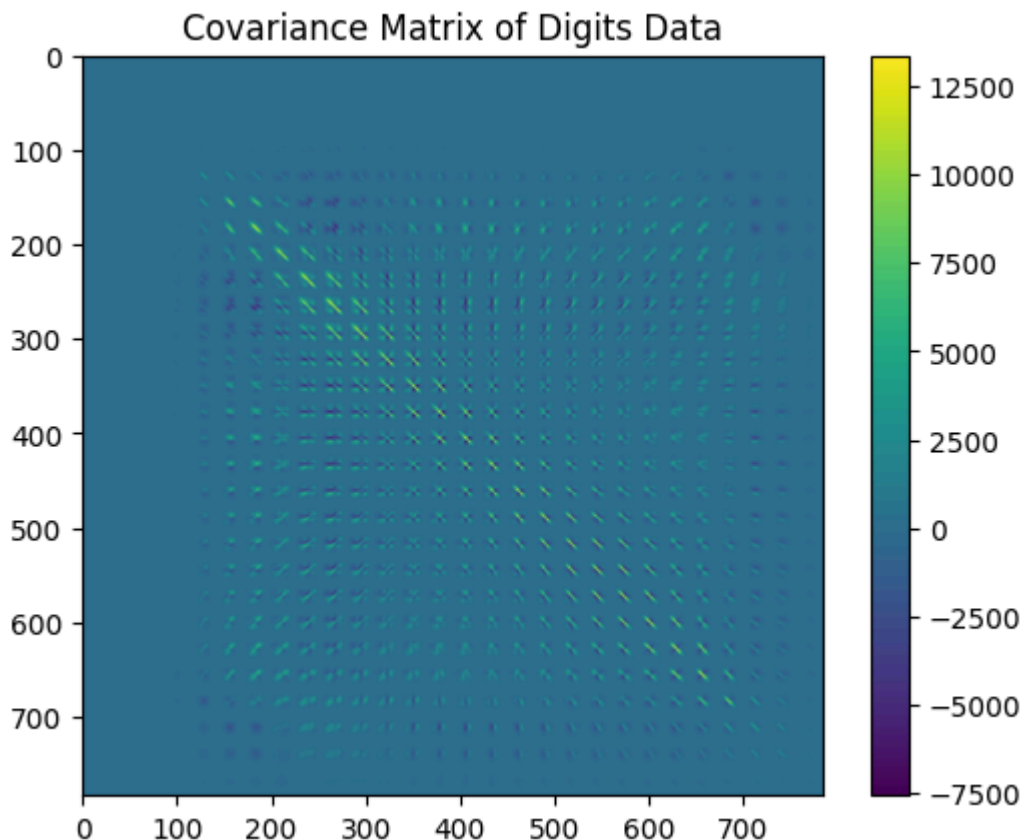
Covariance Matrix Analysis

```
In [ ]: n_features_digits = len(X_digits[0])
```

```
In [ ]: cov_digits = np.cov(X_digits.T)
cov_digits
```

```
Out[ ]: array([[0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               ...,
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.]], shape=(784, 784))
```

```
In [ ]: plt.figure()
display_cov(cov_digits, "Digits Data")
plt.show()
```



```
In [ ]: np.max(cov_temp)
```

```
Out [ ]: np.float64(4.532380952380951)
```

The data at the origin of this covariance matrix is a flattened version of the 28*28 images. From this image we can observe that border pixels are all uncorrelated and that the correlated pixels are concentrated at the center of the image. We can observe that the absolute value of these center pixels on the x axis is progressively increases as y increases and we get at the center of the images then proceeds to diminish until it gets to a constant value of 0.

Each apparent sub-matrix corresponds to the covariance of a line with another (characterized by their position on the y axis). Their form vary but represent a coherent evolution according to their position on the matrix. For example the pixels from the lines 4 to 13 (top of the image), seems to be strongly correlated

Let's observe more precisely the covariance of pixels on one line of the images :

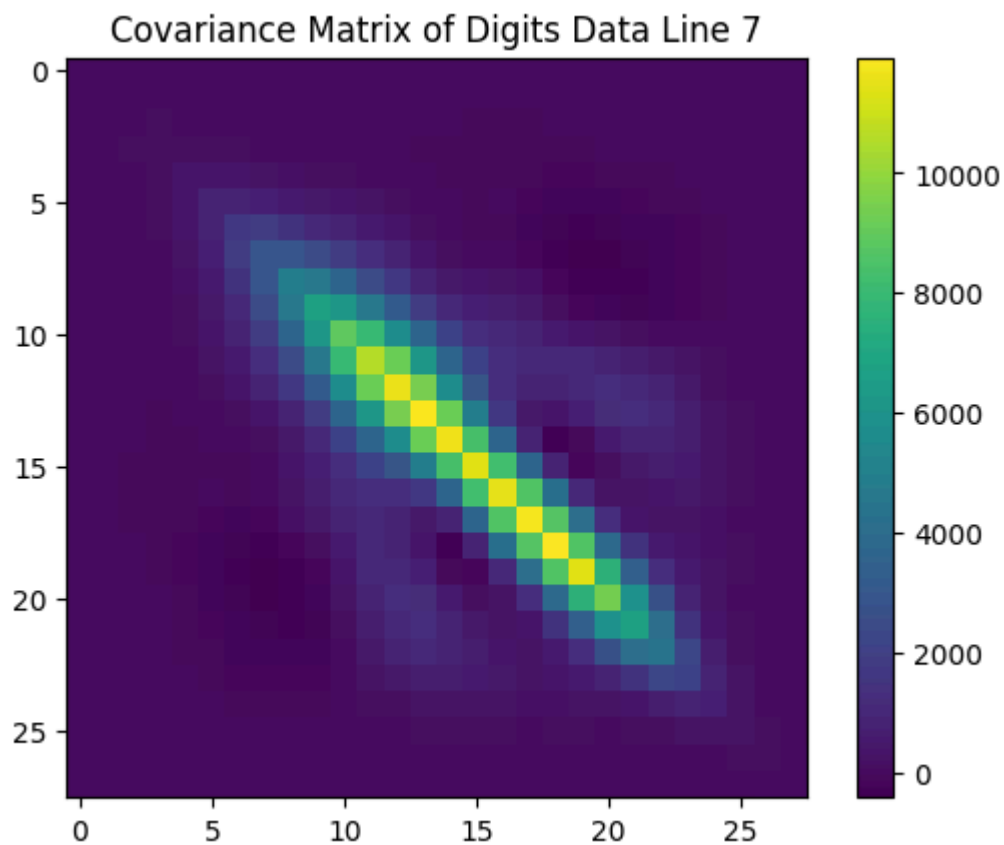
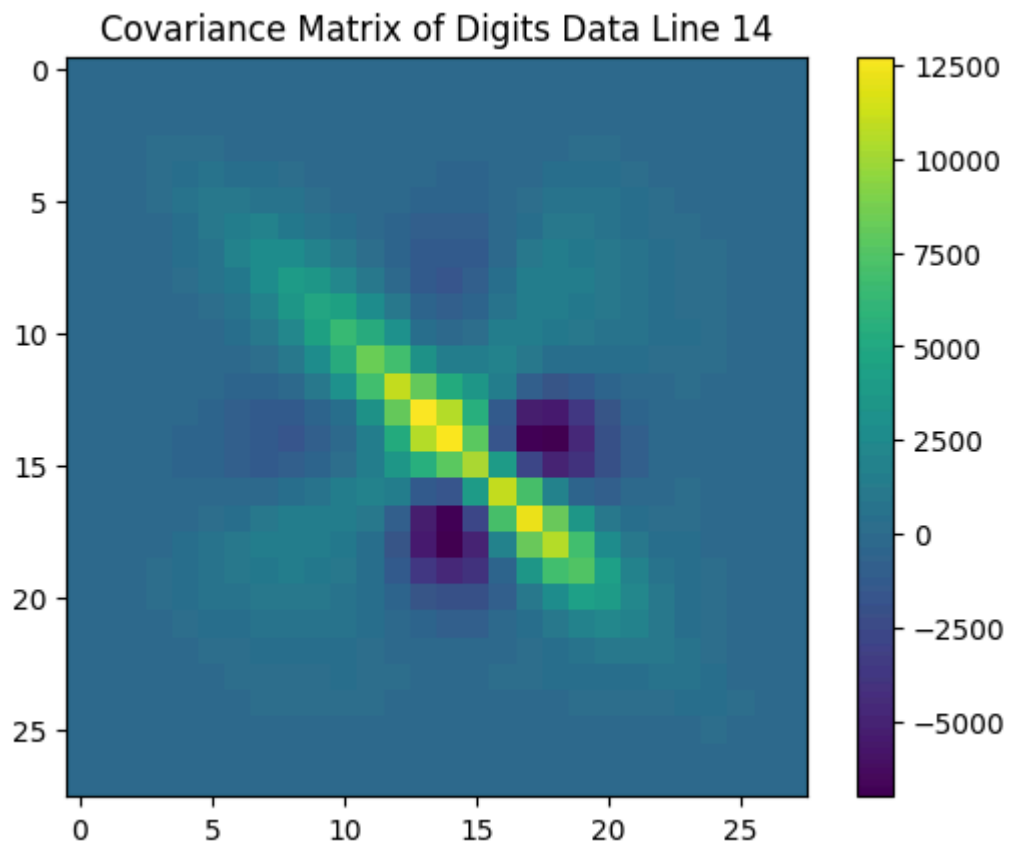
```
In [ ]: ## Covariance of one line with itself

plt.figure()
display_cov(cov_digits[n_features_digits//2 : n_features_digits//2 + 28,
plt.show()

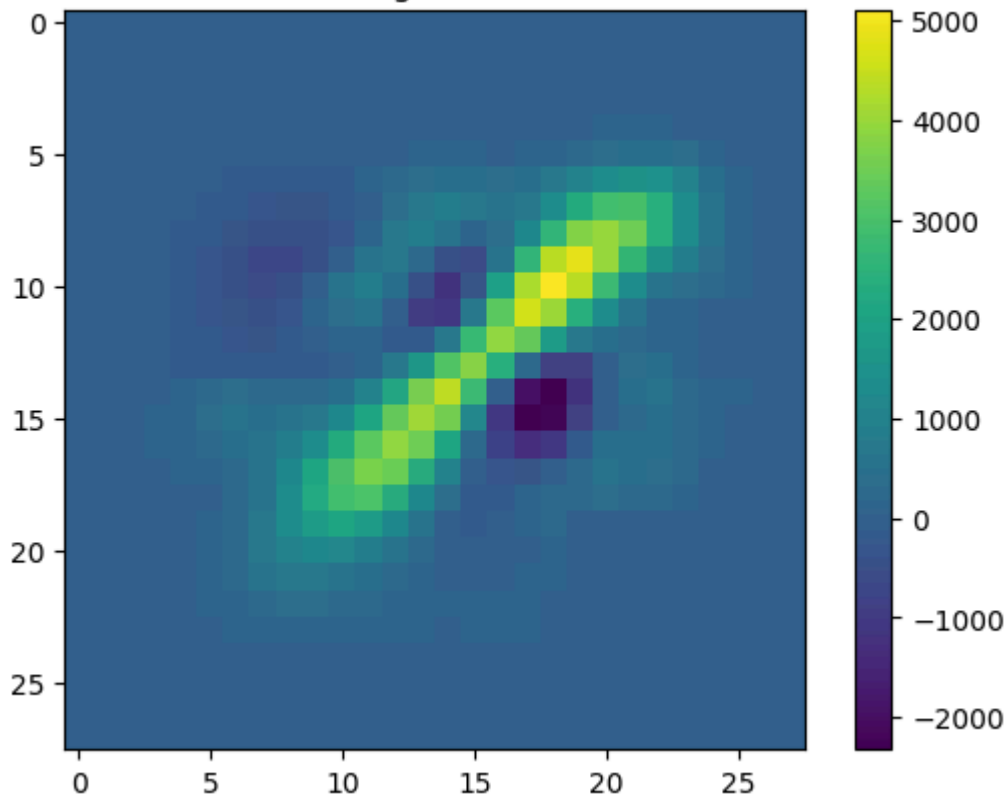
plt.figure()
display_cov(cov_digits[n_features_digits//4 : n_features_digits//4 + 28,
plt.show()

## Covariance of two different lines

plt.figure()
display_cov(cov_digits[3*n_features_digits//4 : 3*n_features_digits//4 +
```



Covariance Matrix of Digits Data Line 21 with Line 7



- In the same line, pixels seems to have a strong covariance with their neighbors. It can be explained by the fact that each drawn number has a thickness of a few pixels. It is especially the case at the top of the numbers, 1, 7, and 8 have a more or less large band at their top (in average it's large). Indeed we can observe this strong neighbor covariance for the top lines on the global covariance matrix.
- In different lines, position on the x axis seems to have a strong covariance with their symmetric according to the middle point of the image.

We can also observe this symmetry on the global covariance matrix : as y increase the 'vector' drawn by the pixels with strong covariance progressively rotate :

```
In [ ]: ## For example if we look at the covariance of the line Y = 9 with all th
Y = 9
img_size = 28

fig, axs = plt.subplots(2,14, figsize = (20, 6))

vmin = cov_digits.min()
vmax = cov_digits.max()

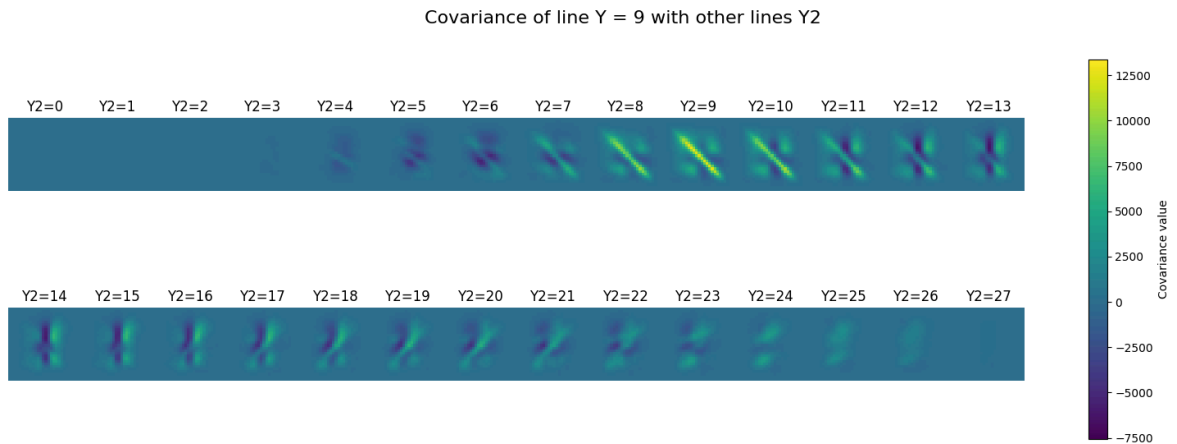
ims = []

for i in range(n_features_digits//img_size):
    im = axs[i//14, i%14].imshow(cov_digits[img_size * Y : img_size * (Y+
axs[i//14, i%14].axis('off')
axs[i//14, i%14].set_title(f"Y2={i}")
    ims.append(im)

plt.subplots_adjust(wspace=0, hspace=0)
```

```
cbar = fig.colorbar(ims[0], ax=axis)
cbar.set_label("Covariance value")

fig.suptitle(f"Covariance of line Y = {Y} with other lines Y2", fontsize
plt.show()
```



This central symmetry can be explained by the form of the numbers 1, 7 and 8 : 8 is completely central symmetric, 1 is almost, 7 is not but contains more or less a diagonal. In average, these numbers admits approximatively a central symmetry.

PCA

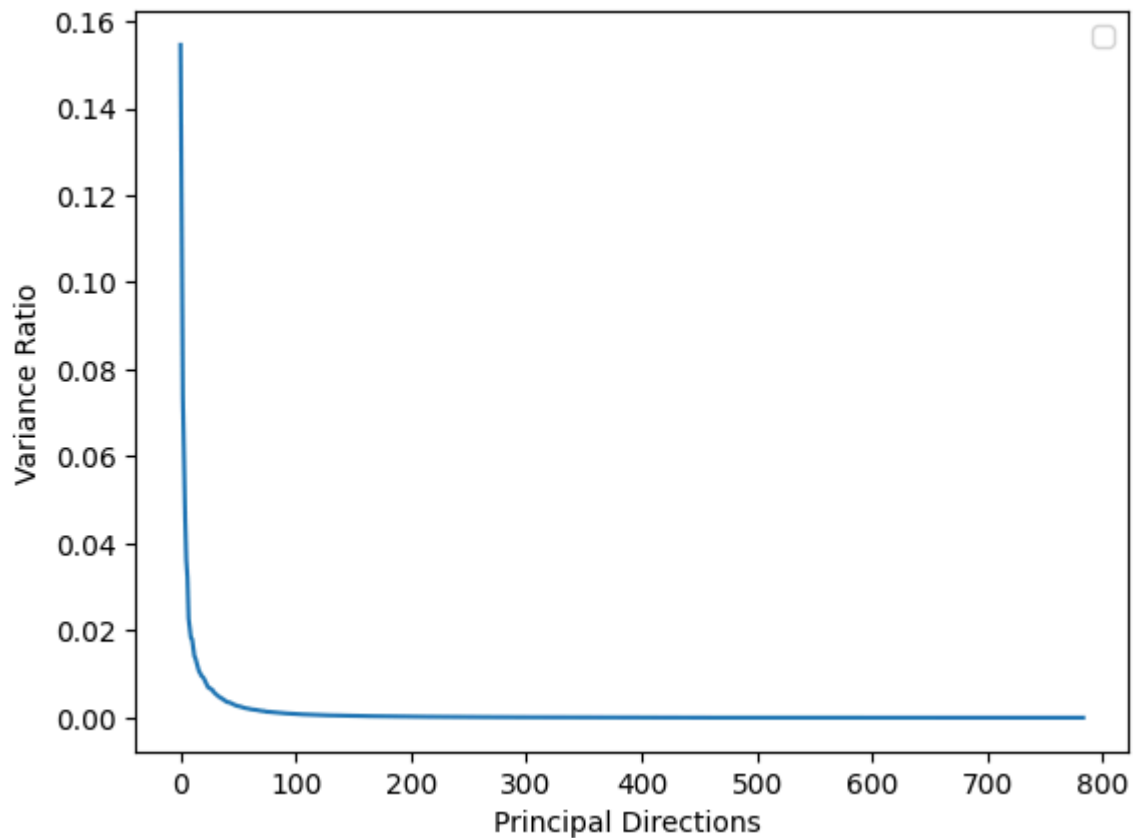
```
In [ ]: pca_digits = PCA(random_state=42)
pca_digits.fit(X_digits)
```

```
Out [ ]: PCA
Parameters
```

```
In [ ]: ## Display Explained Variance Ratio

plt.figure()
plt.plot(pca_digits.explained_variance_ratio_)
plt.xlabel("Principal Directions ")
plt.ylabel("Variance Ratio")
plt.legend()
plt.show()
```

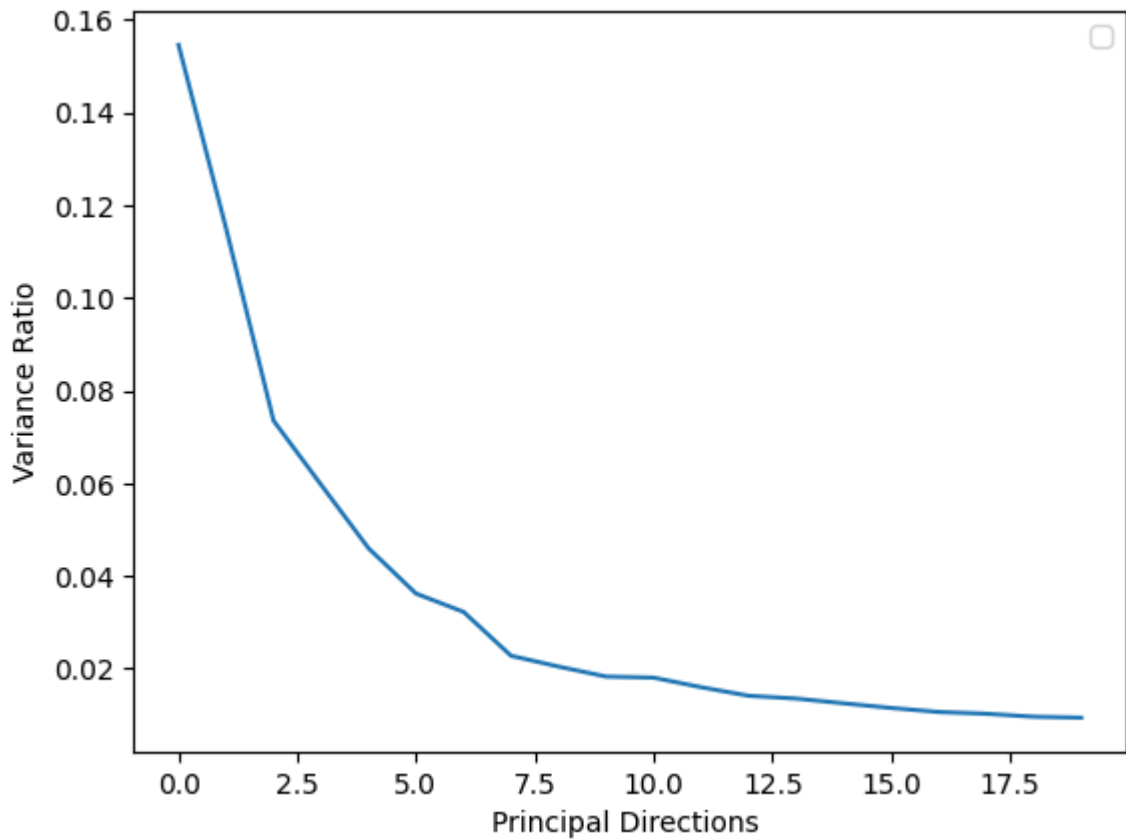
```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/58346818
0.py:7: UserWarning: No artists with labels found to put in legend. Note
that artists whose label start with an underscore are ignored when legend
() is called with no argument.
plt.legend()
```



In []: *## Display Explained Variance Ratio*

```
plt.figure()
plt.plot(pca_digits.explained_variance_ratio_[:20])
plt.xlabel("Principal Directions ")
plt.ylabel("Variance Ratio")
plt.legend()
plt.show()
```

/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/3512128749.py:7: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend()



```
In [ ]: pca_digits.explained_variance_ratio_[:2].sum()
```

```
Out[ ]: np.float64(0.2695057794431238)
```

The quantity of information present on each dimension diminishes exponentially, but if we limit the number of components to 2 the quantity of info retained is 27%.

According to these numbers the PCA with two components seem much less efficient on the digits data than on the temperature data

```
In [ ]: pca_digits_2 = PCA(2, random_state=42)
X_digits_pca_2 = pca_digits_2.fit_transform(X_digits)
X_digits_pca_2
```

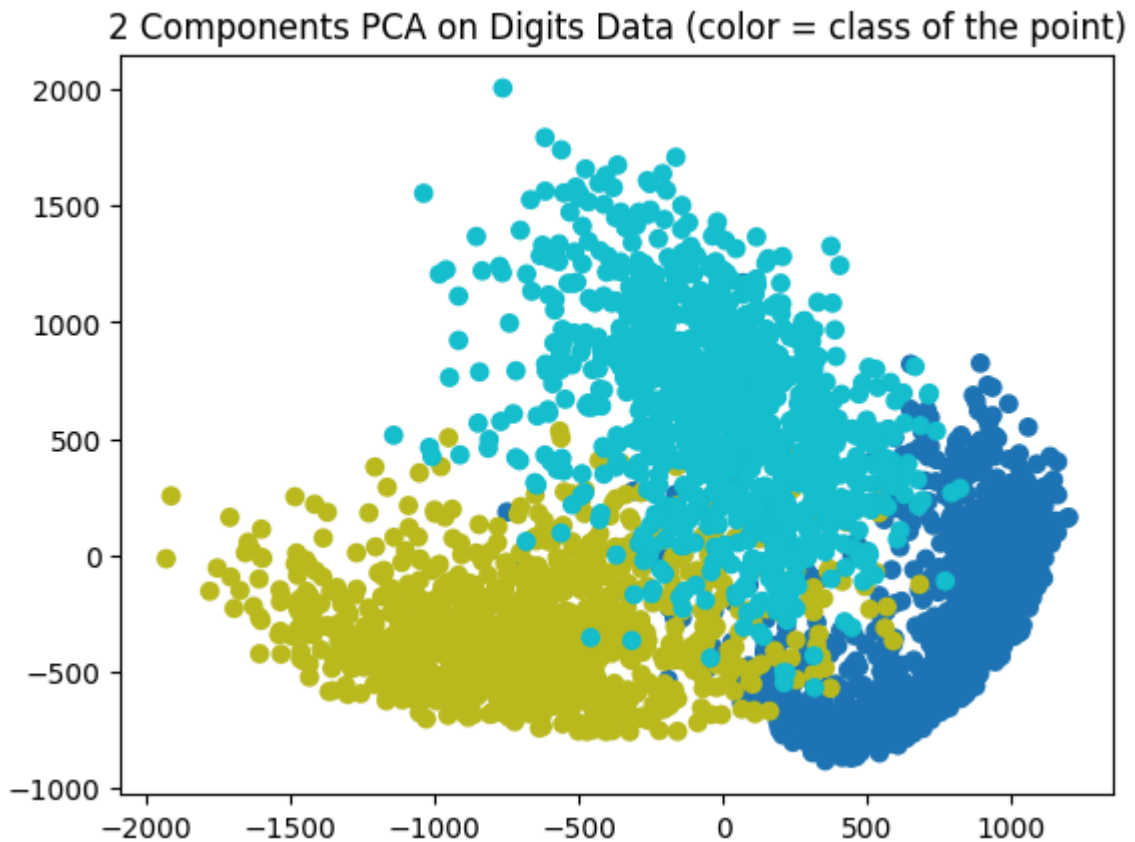
```
Out[ ]: array([[ 970.11884733,  185.68029991],
               [ 210.37122024, -553.13156579],
               [ 301.25509461, -753.20075765],
               ...,
               [ 105.03441629,  190.8572144 ],
               [-616.49810385,  820.0976308 ],
               [-613.73910043,  790.44295574]], shape=(3000, 2))
```

```
In [ ]: X_digits_pca_2[:, 1]
```

```
Out[ ]: array([ 185.68029991, -553.13156579, -753.20075765, ...,  190.8572144 ,
               820.0976308 ,  790.44295574], shape=(3000,))
```

```
In [ ]: colormap = ['blue', 'orange', 'green']

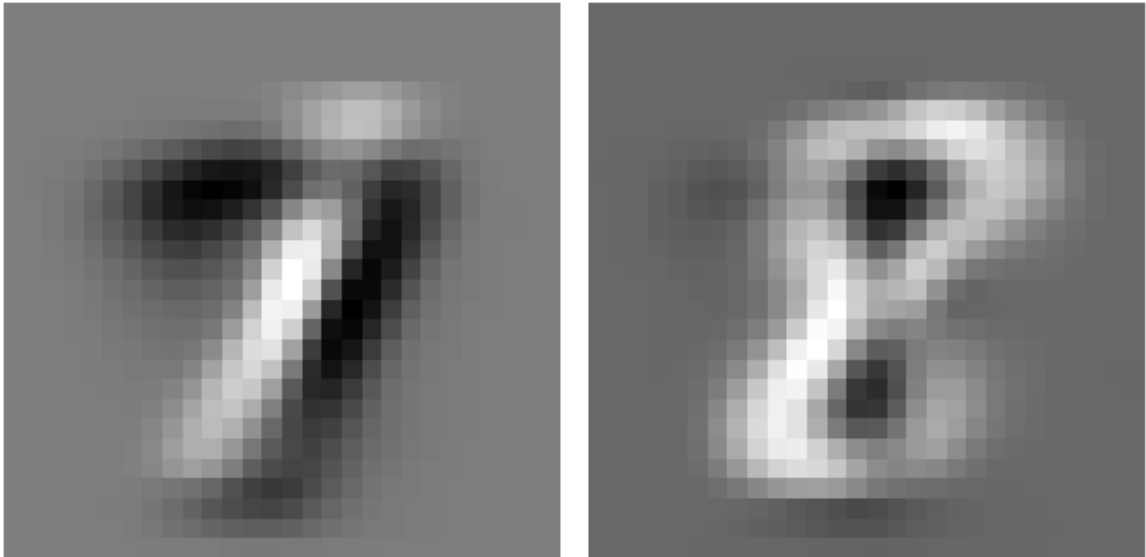
plt.figure()
plt.scatter(X_digits_pca_2[:, 0], X_digits_pca_2[:, 1], c = labels_digits)
plt.title("2 Components PCA on Digits Data (color = class of the point)")
plt.show()
```



We can observe that the points seem clustered by class, but these clusters are not separated enough to differentiate them without color markers.

```
In [ ]: digits_pca_components = pca_digits_2.components_
plt.figure()
display_images(digits_pca_components)
plt.show()
```

<Figure size 640x480 with 0 Axes>



Each of the components seems to have the effect of drawing a specific number while deleting an other

- Moving along the first direction draws a 1 and delete a 7
- Moving along the second direction draws an 8 and delete a 1

```
In [ ]: c = 2
nb_compos_digits = []

while c < n_features_digits:
    nb_compos_digits.append(c)
    c *= 2
nb_compos_digits.append(None)
nb_compos_digits
```

```
Out[ ]: [2, 4, 8, 16, 32, 64, 128, 256, 512, None]
```

```
In [ ]: recon_digits = []
for n in nb_compos_digits :
    pca_temp = PCA(n, random_state=42)
    data_temp = pca_temp.fit_transform(X_digits)
    r = pca_temp.inverse_transform(data_temp)
    recon_digits.append(r)
vmin = np.array(recon_digits).min()
vmax = np.array(recon_digits).max()
```

```
In [ ]: nb_samples = 10
random_index = np.random.randint(0, 3000, 10)
```

```
In [ ]: fig, axs = plt.subplots(nb_samples, len(nb_compos_digits), figsize = (9,1)
for i in range(len(recon_digits)) :
    r = recon_digits[i]
    for j in range(nb_samples):
        axs[i,j].imshow(r[random_index[j]].reshape((28,28)), cmap = 'gray')
        axs[i,j].axis('off')
    axs[i, 0].set_title(f"n = {nb_compos_digits[i]}", fontsize=14, va='center')

plt.legend()
plt.suptitle("Reconstituted Samples from PCA with Varying Number of Compon")
```

```
plt.tight_layout()
```

```
plt.show()
```

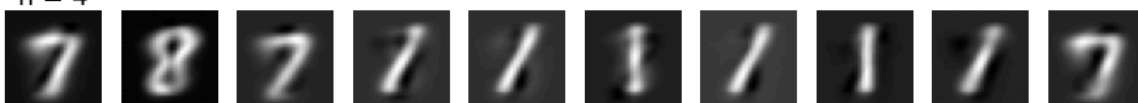
```
/var/folders/5m/n_k44ywj4w37d31x3c70zm8c0000gn/T/ipykernel_72941/73725188
3.py:9: UserWarning: No artists with labels found to put in legend. Note
that artists whose label start with an underscore are ignored when legend
() is called with no argument.
  plt.legend()
```

Reconstituted Samples from PCA with Varying Number of Components n

n = 2



n = 4



n = 8



n = 16



n = 32



n = 64



n = 128



n = 256



n = 512



n = None



With a low number of components the generated images are blurry, especially for a very low number of component where it can be difficult to identify the number drawn. Until n = 64 images of a number can contain a translucent trace of an other

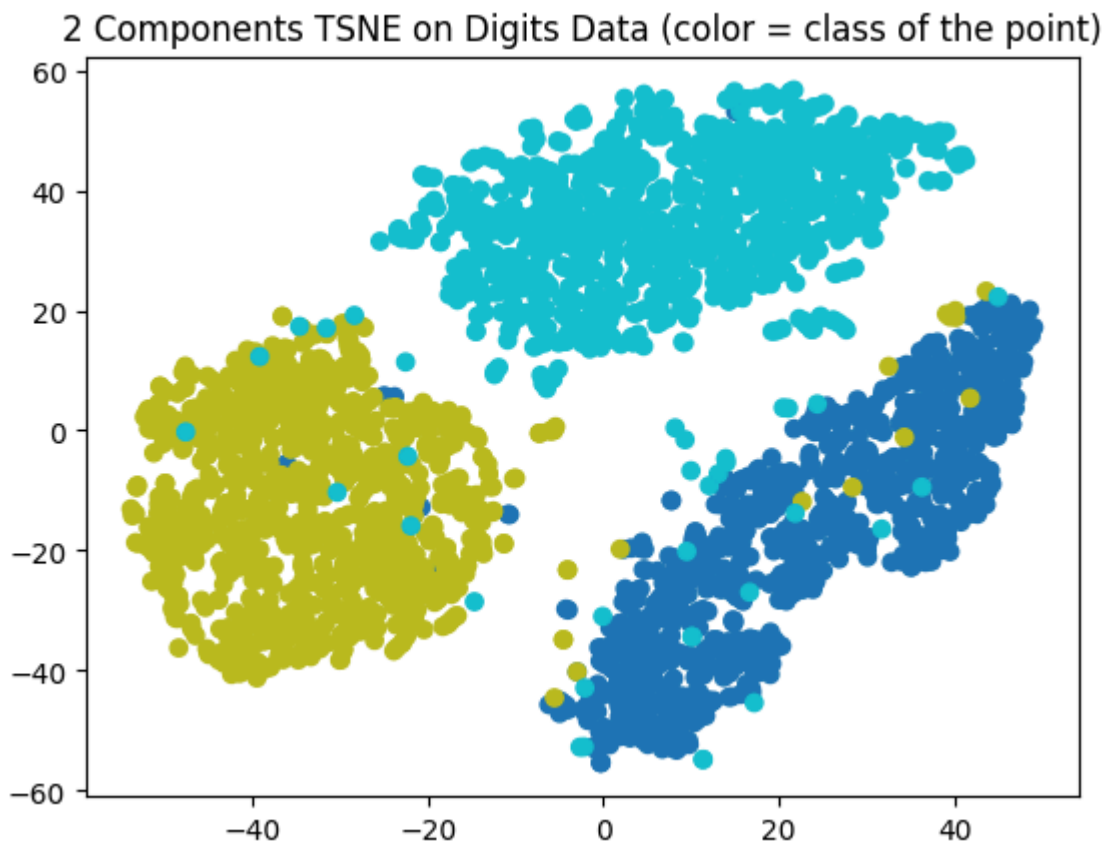
number. For a high number of component the number generated is clearly identifiable although the line drawing it is sharper than in the original data.

TSNE

```
In [ ]: tsne_digits = TSNE(2, random_state=42)
X_digits_tsne = tsne_digits.fit_transform(X_digits)
```

```
In [ ]: colormap = ['blue', 'orange', 'green']

plt.figure()
plt.scatter(X_digits_tsne[:, 0], X_digits_tsne[:, 1], c = labels_digits_G
plt.title("2 Components TSNE on Digits Data (color = class of the point)"
plt.show()
```



The data seems approximatively well separated according to the class, but some errors are still present

```
In [ ]: ppl = np.array([5, 10, 15, 30, 50, 100, 250, 500, 1000, 2000])
X_digits_tsnes = []

for p in ppl :
    tsne_temp = TSNE(2, random_state=42, perplexity= p)
    X_digits_tsnes.append(tsne_temp.fit_transform(X_digits))
```

```
In [ ]: X_digits_tsnes[i][:,0]
```

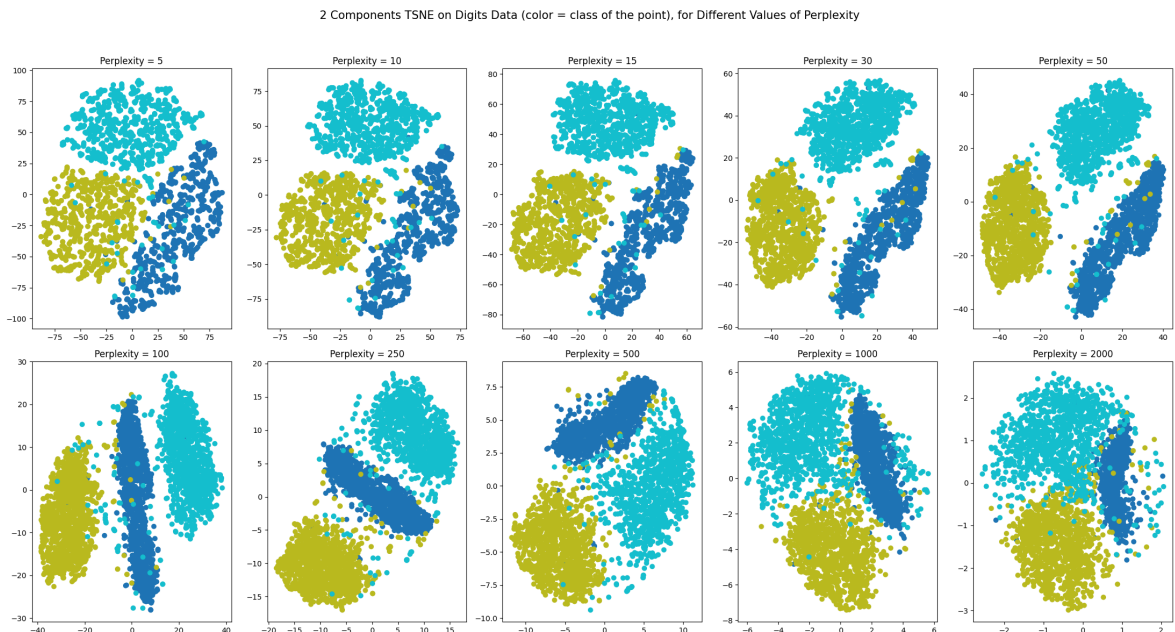
```
Out[ ]: array([ 0.9259401 ,  1.0297534 ,  0.8080825 , ..., -0.46065307,
                -2.1332216 , -1.6222111 ], shape=(3000,), dtype=float32)
```

```
In [ ]: fig, axs = plt.subplots(2,5, figsize = (22,12))

for i in range(len(ppl)) :

    data = X_digits_tsnes[i]
    a = axs[i//5,i%5]
    a.scatter(data[:, 0], data[:, 1], c = labels_digits_GT, cmap = 'tab10')
    a.set_title(f"Perplexity = {ppl[i]}")

plt.suptitle(f"2 Components TSNE on Digits Data (color = class of the poi
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```



As we could expect, the TSNE is more adapted than PCA to discriminate drawn numbers, as their line is not linear (notably the 8). For a particularly low or high perplexity (near 0 or 3000) the clusters are not clearly separated, the data from different class is even mixed up for very high values.